

ARTICLE

<https://doi.org/10.1038/s41467-020-17236-y>

OPEN

A solution to the learning dilemma for recurrent networks of spiking neurons

Guillaume Bellec^{1,2}, Franz Scherr^{1,2}, Anand Subramoney¹, Elias Hajek¹, Darjan Salaj¹, Robert Legenstein¹ & Wolfgang Maass¹✉

Recurrently connected networks of spiking neurons underlie the astounding information processing capabilities of the brain. Yet in spite of extensive research, how they can learn through synaptic plasticity to carry out complex network computations remains unclear. We argue that two pieces of this puzzle were provided by experimental data from neuroscience. A mathematical result tells us how these pieces need to be combined to enable biologically plausible online network learning through gradient descent, in particular deep reinforcement learning. This learning method—called e-prop—approaches the performance of back-propagation through time (BPTT), the best-known method for training recurrent neural networks in machine learning. In addition, it suggests a method for powerful on-chip learning in energy-efficient spike-based hardware for artificial intelligence.

¹Institute of Theoretical Computer Science, Graz University of Technology, Inffeldgasse 16b, Graz, Austria. ²These authors contributed equally: Guillaume Bellec, Franz Scherr. ✉email: maass@igi.tugraz.at

Networks of neurons in the brain differ in at least two essential aspects from deep neural networks in machine learning: they are recurrently connected, forming a giant number of loops, and they communicate via asynchronously emitted stereotypical electrical pulses, called spikes, rather than bits or numbers that are produced in a synchronized manner by each layer of a deep feedforward network. We consider the arguably most prominent model for spiking neurons in the brain: leaky integrate-and-fire (LIF) neurons, where spikes that arrive from other neurons through synaptic connections are multiplied with the corresponding synaptic weight, and are linearly integrated by a leaky membrane potential. The neuron fires—i.e., emits a spike—when the membrane potential reaches a firing threshold.

But it is an open problem how recurrent networks of spiking neurons (RSNNs) can learn, i.e., how their synaptic weights can be modified by local rules for synaptic plasticity so that the computational performance of the network improves. In deep learning, this problem is solved for feedforward networks through gradient descent for a loss function E that measures imperfections of current network performance¹. Gradients of E are propagated backwards through all layers of the feedforward network to each synapse through a process called backpropagation. Recurrently connected networks can compute more efficiently because each neuron can participate several times in a network computation, and they are able to solve tasks that require integration of information over time or a non-trivial timing of network outputs according to task demands. Therefore, the impact of a synaptic weight on the loss function (see Fig. 1a) is more indirect, and learning through gradient descent becomes substantially more difficult. This problem is aggravated if there are slowly changing hidden variables in the neuron model, as in neurons with spike-frequency adaptation (SFA). Neurons with SFA are quite common in the neocortex², and it turns out that their inclusion in the RSNN significantly increases the computational power of the network³. In fact, RSNNs trained through gradient descent acquire then similar computing capabilities as networks of LSTM (long short-term memory) units, the state of the art for recurrent neural networks in machine learning. Because of this functional relation to LSTM networks these RSNN models are referred to as LSNNs³.

In machine learning, one trains recurrent neural networks by unrolling the network into a virtual feedforward network¹, see Fig. 1b, and applying the backpropagation algorithm to that (Fig. 1c). This method is called backpropagation through time (BPTT), as it requires propagation of gradients backwards in time.

With a careful choice of the pseudo derivative for handling the discontinuous dynamics of spiking neurons, one can apply BPTT also to RSNNs, and RSNNs were able to learn in this way to solve really demanding computational tasks^{3,4}. But the dilemma is that BPTT requires storing the intermediate states of all neurons during a network computation, and merging these in a subsequent offline process with gradients that are computed backwards in time (see Fig. 1c). This makes it very unlikely that BPTT is used by the brain⁵.

The previous lack of powerful online learning methods for RSNNs also affected the use of neuromorphic computing hardware, which aims at a drastic reduction in the energy consumption of AI implementations. A substantial fraction of this neuromorphic hardware, such as SpiNNaker⁶ or Intel's Loihi chip⁷, implements RSNNs. Although it does not matter here whether the learning algorithm is biologically plausible, the excessive storage and offline processing demands of BPTT make this option unappealing. Hence, there also exists a learning dilemma for RSNNs in neuromorphic hardware.

We are not aware of previous work on online gradient descent learning methods for RSNNs, neither for supervised learning nor for reinforcement learning (RL). There exists, however, preceding work on online approximations of gradient descent for non-spiking neural networks based on⁸, which we review in the Discussion Section.

Two streams of experimental data from neuroscience provide clues about the organization of online network learning in the brain:

First, neurons in the brain maintain traces of preceding activity on the molecular level, for example, in the form of calcium ions or activated CaMKII enzymes⁹. In particular, they maintain a fading memory of events where the presynaptic neuron fired before the postsynaptic neuron, which is known to induce synaptic plasticity if followed by a top-down learning signal^{10–12}. Such traces are often referred to as eligibility traces.

Second, in the brain, there exists an abundance of top-down signals such as dopamine, acetylcholine, and neural firing¹³ related to the error-related negativity, that inform local populations of neurons about behavioral results. Furthermore, dopamine signals^{14,15} have been found to be specific for different target populations of neurons, rather than being global. We refer in our learning model to such top-down signals as learning signals.

A re-analysis of the mathematical basis of gradient descent learning in recurrent neural networks tells us how local eligibility traces and top-down learning signals should be optimally combined—without requiring backpropagation of signals through time. The resulting learning method e-prop is illustrated in Fig. 1d. It learns slower than BPTT, but tends to approximate the performance of BPTT, thereby providing a first solution to the learning dilemma for RSNNs. Furthermore, e-prop also works for RSNNs with more complex neuron models, such as LSNNs. This new learning paradigm elucidates how the brain could learn to recognize phonemes in spoken language, solve temporal credit assignment problems, and acquire new behaviors just from rewards.

Results

Mathematical basis for e-prop. Spikes are modeled as binary variables z_j^t that assume value 1 if neuron j fires at time t , otherwise value 0. It is common in models to let t vary over small discrete time steps, e.g., of 1 ms length. The goal of network learning is to find synaptic weights W that minimize a given loss function E . E may depend on all or a subset of the spikes in the network. E measures in the case of regression or classification learning the deviation of the actual output y_k^t of each output neuron k at time t from its given target value $y_k^{*,t}$ (Fig. 1a). In RL, the goal is to optimize the behavior of an agent in order to maximize obtained rewards. In this case, E measures deficiencies of the current agent policy to collect rewards.

The gradient $\frac{dE}{dW_{ji}}$ for the weight W_{ji} of the synapse from neuron i to neuron j tells us how this weight should be changed in order to reduce E . It can in principle be estimated—in spite of the fact that the implicit discrete variable z_j^t is non-differentiable—with the help of a suitable pseudo derivative for spikes as in refs. ^{3,4}. The key innovation is a rigorous proof (see “Methods”) that the gradient $\frac{dE}{dW_{ji}}$ can be represented as a sum of products over the time steps t of the RSNN computation, where the second factor is just a local gradient that does not depend on E :

$$\frac{dE}{dW_{ji}} = \sum_t \frac{dE}{dz_j^t} \cdot \left[\frac{dz_j^t}{dW_{ji}} \right]_{\text{local}} \quad (1)$$

This local gradient is defined as a sum of products of partial derivatives concerning the hidden state $\mathbf{h}_j^t$ of neuron j at time t

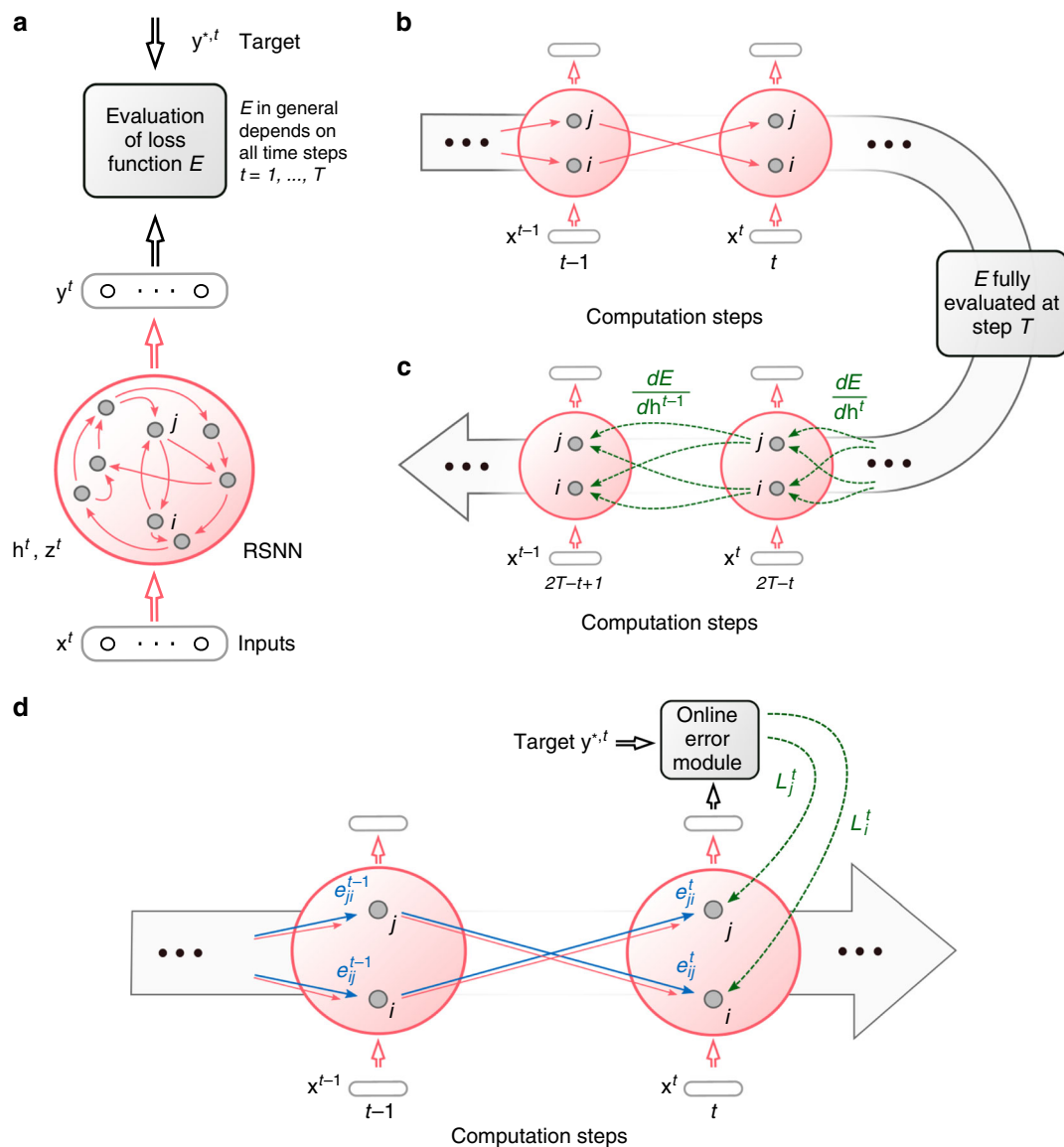


Fig. 1 Schemes for BPTT and e-prop. **a** RSNN with network inputs $\mathbf{x}$, neuron spikes $\mathbf{z}$, hidden neuron states $\mathbf{h}$, and output targets $\mathbf{y}^*$, for each time step t of the RSNN computation. Output neurons $\mathbf{y}$ provide a low-pass filter of a weighted sum of network spikes $\mathbf{z}$. **b** BPTT computes gradients in the unrolled version of the network. It has a new copy of the neurons of the RSNN for each time step t . A synaptic connection from neuron i to neuron j of the RSNN is replaced by an array of feedforward connections, one for each time step t , which goes from the copy of neuron i in the layer for time step t to a copy of neuron j in the layer for time step $t + 1$. All synapses in this array have the same weight: the weight of this synaptic connection in the RSNN. **c** Loss gradients of BPTT are propagated backwards in time and retrograde across synapses in an offline manner, long after the forward computation has passed a layer. **d** Online learning dynamics of e-prop. Feedforward computation of eligibility traces is indicated in blue. These are combined with online learning signals according to Eq. (1).

and preceding time steps, which can be updated during the forward computation of the RNN by a simple recursion (Eq. (14)). This term $\left[\frac{dz_j^t}{dW_{ji}} \right]_{\text{local}}$ is not an approximation. Rather, it collects the maximal amount of information about the network gradient $\frac{dE}{dW_{ji}}$ that can be computed locally in a forward manner. Therefore, it is the key-factor of e-prop. As it reduces for simple neuron models—whose internal state is fully captured by its membrane potential—to a variation of terms that are commonly referred to as eligibility traces for synaptic plasticity¹², we also refer to

$$e_{ji}^t \stackrel{\text{def}}{=} \left[\frac{dz_j^t}{dW_{ji}} \right]_{\text{local}} \quad (2)$$

as eligibility trace. But most biological neurons have additional hidden variables that change on a slower time scale, such as the firing threshold of a neuron with firing threshold adaptation. Furthermore, these slower processes in neurons are essential for attaining with spiking neurons similarly powerful computing capabilities as LSTM networks³. Hence, the form that this eligibility trace e_{ji}^t takes for adapting neurons (see Eq. (25)) is essential for understanding e-prop, and it is the main driver behind the resulting qualitative jump in computing capabilities of RSNNs, which are attainable through biologically plausible learning. Eqs. (1) and (2) yield the representation

$$\frac{dE}{dW_{ji}} = \sum_t L_j^t e_{ji}^t \quad (3)$$

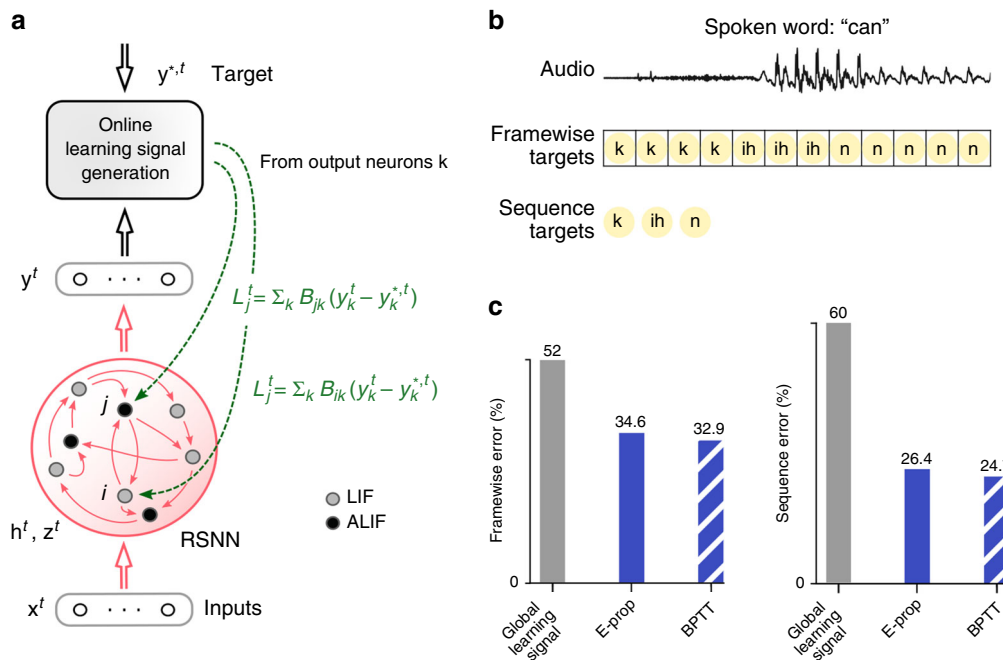


Fig. 2 Comparison of BPTT and e-prop for learning phoneme recognition. **a** Network architecture for e-prop, illustrated for an LSNN consisting of LIF and ALIF neurons. **b** Input and target output for the two versions of TIMIT. **c** Performance of BPTT and symmetric e-prop for LSNNs consisting of 800 neurons for frame-wise targets and 2400 for sequence targets (random and adaptive e-prop produced similar results, see Supplementary Fig. 2). To obtain the Global learning signal baselines, the neuron-specific feedbacks are replaced with global ones.

of the loss gradient, where we refer to $L_j^t \stackrel{\text{def}}{=} \frac{dE}{dz_j^t}$ as the learning signal for neuron j . This equation defines a clear program for approximating the network loss gradient through local rules for synaptic plasticity: change each weight W_{ji} at step t proportionally to $-L_j^t e_{ji}^t$, or accumulate these so-called tags in a hidden variable that is translated occasionally into an actual weight change. Hence, e-prop is an online learning method in a strict sense (see Fig. 1d). In particular, there is no need to unroll the network as for BPTT.

As the ideal value $\frac{dE}{dz_j^t}$ of the learning signal L_j^t also captures influences that the current spike output z_j^t of neuron j may have on E via future spikes of other neurons, its precise value is in general not available at time t . We replace it by an approximation, such as $\frac{\partial E}{\partial z_j^t}$, which ignores these indirect influences (this partial derivative $\frac{\partial E}{\partial z_j^t}$ is written with a rounded ∂ to signal that it captures only the direct influence of the spike z_j^t on the loss function E). This approximation takes only currently arising losses at the output neurons k of the RSNN into account, and routes them with neuron-specific weights B_{jk} to the network neurons j (see Fig. 2a):

$$L_j^t = \sum_k B_{jk} \underbrace{(y_k^t - y_k^{*,t})}_{\text{deviation of output } k \text{ at time } t}. \quad (4)$$

Although this approximate learning signal L_j^t only captures errors that arise at the current time step t , it is combined in Eq. (3) with an eligibility trace e_{ji}^t that may reach far back into the past of neuron j (see Fig. 3b), thereby alleviating the need to solve the temporal credit assignment problem by propagating signals backwards in time (like in BPTT).

There are several strategies for choosing the weights B_{jk} for this online learning signal. In symmetric e-prop, we set it equal to the corresponding weight W_{kj}^{out} of the synaptic connection from

neuron j to output neuron k , as demanded by $\frac{\partial E}{\partial z_j^t}$. Note that this learning signal would actually implement $\frac{dE}{dz_j^t}$ exactly in the absence of recurrent connections in the network. Biologically more plausible are two variants of e-prop that avoid weight sharing: in random e-prop, the values of all weights B_{jk} —even for neurons j that are not synaptically connected to output neuron k —are randomly chosen and remain fixed, similar to Broadcast Alignment for feedforward networks^{16–18}. In adaptive e-prop, in addition to using random backward weights, we also let B_{jk} evolve through a simple local plasticity rule that mirrors the plasticity rule applied to W_{kj}^{out} for neurons j that are synaptically connected to output neuron k (see Supplementary Note 2).

Resulting synaptic plasticity rules (see Methods) look similar to previously proposed plasticity rules¹² for the special case of LIF neurons without slowly changing hidden variables. In particular, they involve postsynaptic depolarization as one of the factors, similarly as the data-based Clopath-rule in ref. 19, see Supplementary Note 6 for an analysis.

Learning phoneme recognition with e-prop. The phoneme recognition task TIMIT²⁰ is one of the most commonly used benchmarks for temporal processing capabilities of different types of recurrent neural networks and different learning approaches²¹. It comes in two versions. Both use, as input, acoustic speech signals from sentences that are spoken by 630 speakers from eight dialect regions of the USA (see the top of Fig. 2b for a sample segment). In the simpler version, used for example in ref. 21, the goal is to recognize which of 61 phonemes is spoken in each 10 ms time frame (frame-wise classification). In the more sophisticated version from ref. 22, which achieved an essential step toward human-level performance in speech-to-text transcription, the goal is to recognize the sequence of phonemes in the entire spoken sentence independently of their timing (sequence transcription). RSNNs consisting only of LIF neurons do not even reach good performance on TIMIT with BPTT³.

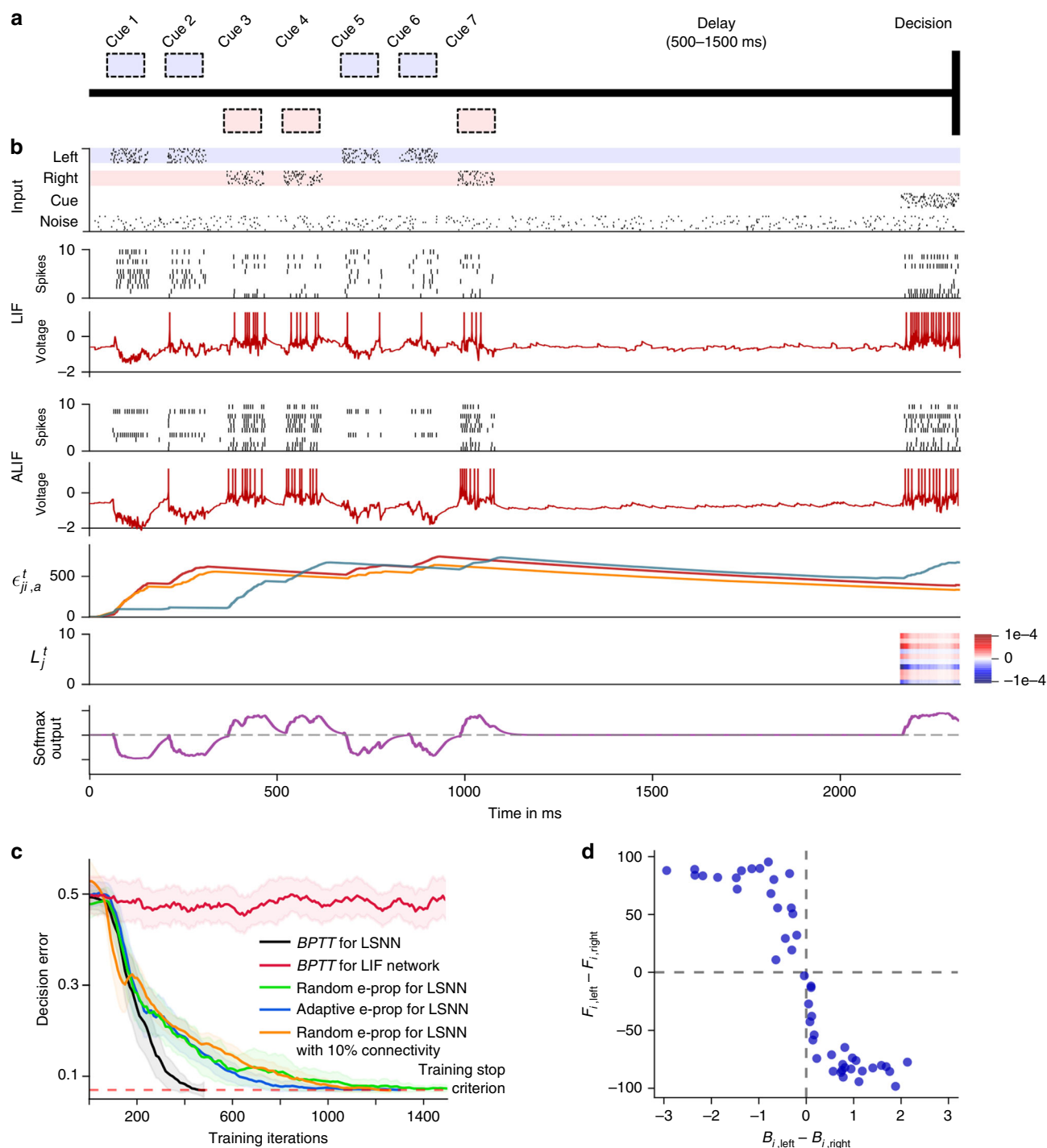


Fig. 3 Solving a task with difficult temporal credit assignment. **a** Setup of corresponding rodent experiments of ref. ²³ and ref. ¹⁴, see Supplementary Movie 1. **b** Input spikes, spiking activity of 10 out of 50 sample LIF neurons and 10 out of 50 sample ALIF neurons, membrane potentials (more precisely: $v_j^t - A_j^t$) for two sample neurons j , three samples of slow components of eligibility traces, sample learning signals for 10 neurons and softmax network output. **c** Learning curves for BPTT and two e-prop versions applied to LSNNs, and BPTT applied to an RSNN without adapting neurons (red curve). Orange curve shows learning performance of e-prop for a sparsely connected LSNN, consisting of excitatory and inhibitory neurons (Dale's law obeyed). The shaded areas are the 95% confidence intervals of the mean accuracy computed with 20 runs. **d** Correlation between the randomly drawn broadcast weights B_{jk} for $k = \text{left/right}$ for learning signals in random e-prop and resulting sensitivity to left and right input components after learning. $f_{j,\text{left}}$ ($f_{j,\text{right}}$) was the resulting average firing rate of neuron j during presentation of left (right) cues after learning.

Hence, we are considering here LSNNs, where a random subset of the neurons is a variation of the LIF model with firing rate adaptation (adaptive LIF (ALIF) neurons), see Methods. The name LSNN is motivated by the fact that this special case of the

RSNN model can achieve through training with BPTT similar performance as an LSTM network³.

E-prop approximates the performance of BPTT on LSNNs for both versions of TIMIT very well, as shown in Fig. 2c.

Furthermore, LSNNs could solve the framewise classification task without any neuron firing more frequently than 12 Hz (spike count taken over 32 spoken sentences), demonstrating that they operate in an energy-efficient spike-coding—rather than a rate coding—regime. For the more difficult version of TIMIT, we trained as in ref. ²², a complex LSNN consisting of a feedforward sequence of three recurrent networks. Our results show that e-prop can also handle learning for such more complex network structures very well. In Supplementary Fig. 4 we show for comparison also the performance of e-prop and BPTT for LSTM networks on the same tasks. These data show that for both versions of TIMIT the performance of e-prop for LSNNs comes rather close to that of BPTT for LSTM networks. In addition, they show that e-prop also provides for LSTM networks a functionally powerful online learning method.

Solving difficult temporal credit assignment. A hallmark of cognitive computations in the brain is the capability to go beyond a purely reactive mode: to integrate diverse sensory cues over time, and to wait until the right moment arrives for an action. A large number of experiments in neuroscience analyze neural coding after learning such tasks (see e.g., refs. ^{14,23}). But it had remained unknown how one can model the learning of such cognitive computations in RSNNs of the brain. In order to test whether e-prop can solve this problem, we considered the same task that was studied in the experiments of ref. ²³ and ref. ¹⁴. There a rodent moved along a linear track in a virtual environment, where it encountered several visual cues on the left and right, see Fig. 3a and Supplementary Movie 1. Later, when it arrived at a T-junction, it had to decide whether to turn left or right. It was rewarded when it turned to that side from which it had previously received the majority of visual cues. This task is not easy to learn as the subject needs to find out that it does not matter on which side the last cue was, or in which order the cues were presented. Instead, the subject has to learn to count cues separately for each side and to compare the two resulting numbers. Furthermore, the cues need to be processed properly long before a reward is given. We show in Supplementary Fig. 5 that LSNNs can learn this task via e-prop in exactly the same way just from rewards. But as the way how e-prop solves the underlying temporal credit assignment problem is easier to explain for the supervised learning version of this task, we discuss here the case where a teacher tells the subject at the end of each trial what would have been the right decision. This still yields a challenging scenario for any online learning method since non-zero learning signals L_j^t arise only during the last 150 ms of a trial (Fig. 3b). Hence, all synaptic plasticity has to take place during these last 150 ms, long after the input cues have been processed. Nevertheless, e-prop is able to solve this learning problem, see Fig. 3c and Supplementary Movie 2. It just needs a bit more time to reach the same performance level as offline learning via BPTT (see Supplementary Movie 3). Whereas this task cannot even be solved by BPTT with a regular RSNN that has no adapting neurons (red curve in Fig. 3c), all three previously discussed variations of e-prop can solve it if the RSNN contains adapting neurons. We explain in Supplementary Note 2 how this task can also be solved by sparsely connected LSNNs consisting of excitatory and inhibitory neurons: by integrating stochastic rewiring²⁴ into e-prop.

But how can the neurons in the LSNN learn to record and count the input cues if all the learning signals are identically 0 until the last 150 ms of a 2250 ms long trial (see 2nd to last row of Fig. 3b)?

For answering this question, one should note that firing of a neuron j at time t can affect the loss function E at a later time

point $t' > t$ in two different ways: via route (i) it affects future values of slow hidden variables of neuron j (e.g., its firing threshold), which may then affect the firing of neuron j at t' , which in turn may directly affect the loss function at time t' . Via route (ii) it affects the firing of other neurons j' at t' , which directly affects the loss function at time t' .

In symmetric and adaptive e-prop, one uses the partial derivative $\frac{\partial E}{\partial z_j^t}$ as learning signal L_j^t for e-prop—instead of the total derivative $\frac{dE}{dz_j^t}$, which is not available online. This blocks the flow of gradient information along route (ii). But the eligibility trace keeps the flow along route (i) open. Therefore, even symmetric and adaptive e-prop can solve the temporal credit assignment problem of Fig. 3 through online learning: the gradient information that flows along route (i) enables neurons to learn how to process the sensory cues at time points t during the first 1050 ms, although this can affect the loss only at time points $t' > 2100$ ms when the loss becomes non-zero.

This is illustrated in the 3rd last row of Fig. 3b: the slow component $e_{ji,a}^t$ of the eligibility traces e_{ji} of adapting neurons j decays with the typical long time constant of firing rate adaptation (see Eq. (24) and Supplementary Movie 2). As these traces stretch from the beginning of the trial into its last phase, they enable learning of differential responses to left and right input cues that arrived over 1050 ms before any learning signals become non-zero, as shown in the 2nd to last row of Fig. 3b. Hence, eligibility traces provide so-called highways into the future for the propagation of gradient information. These can be seen as biologically realistic replacements for the highways into the past that BPTT employs during its backwards pass (see Supplementary Movie 3).

This analysis also tells us when symmetric e-prop is likely to fail to approximate the performance of BPTT: if the forward propagation of gradients along route (i) cannot reach those later time points t' at which the value of the loss function becomes salient. One can artificially induce this in the experiment of Fig. 3 by adding to the LSNN—which has the standard architecture shown in Fig. 2a—hidden layers of a feedforward SNN through which the communication between the LSNN and the readout neurons has to flow. The neurons j' of these hidden layers block route (i), whereas leaving route (ii) open. Hence, the task of Fig. 3 can still be learnt with this modified network architecture by BPTT, but not by symmetric e-prop, see Supplementary Fig. 8.

Identifying tasks where the performance of random e-prop stays far behind that of BPTT is more difficult, as error signals are sent there also to neurons that have no direct connections to readout neurons. For deep feedforward networks it has been shown in ref. ²⁵ that Broadcast Alignment, as defined in refs. ^{17,18}, cannot reach the performance of Backprop for difficult image classification tasks. Hence, we expect that random e-prop will exhibit similar deficiencies with deep feedforward SNNs on difficult classification tasks. We are not aware of corresponding demonstrations of failures of Broadcast Alignment for artificial RNNs, although they are likely to exist. Once they are found, they will probably point to tasks where random e-prop fails for RSNNs. Currently, we are not aware of any.

Figure 3d provides insight into the functional role of the randomly drawn broadcast weights in random e-prop: the difference of these weights determines for each neuron j whether it learns to respond in the first phase of a trial more to cues from the left or right. This observation suggests that neuron-specific learning signals for RSNNs have the advantage that they can create a diversity of feature detectors for task-relevant network inputs. Hence, a suitable weighted sum of these feature detectors is later able to cancel remaining errors at the network output, similarly as in the case of feedforward networks¹⁶.

We would like to point out that the use of the familiar actor-critic method in reward-based e-prop, which we will discuss in the next section, provides an additional channel by which information about future losses can gate synaptic plasticity of the e-prop learner at the current time step t : through the estimate $V(t)$ of the value of the current state, that is simultaneously learnt via internally generated reward prediction errors.

Reward-based e-prop. Deep RL has significantly advanced the state of the art in machine learning and AI through clever applications of BPTT to RL²⁶. We found that one of the arguably most powerful RL methods within the range of deep RL approaches that are not directly biologically implausible, policy gradient in combination with actor-critic, can be implemented with e-prop. This yields the biologically plausible and hardware friendly deep RL algorithm reward-based e-prop. The LSNN learns here both an approximation to the value function (the critic) and a stochastic policy (the actor). Neuron-specific learning signals are combined in reward-based e-prop with a global signal that transmits reward prediction errors (Fig. 4b). In contrast to the supervised case, where the learning signals L_j^t depend on the deviation from an external target signal, the learning signals communicate here how a stochastically chosen action deviates from the action mean that is currently proposed by the network.

In such RL tasks, the learner needs to explore its environment, and find out which action gets rewarded in what state²⁷. There is no teacher that tells the learner what action would be optimal; in fact, the learner may never find that out. Nevertheless, learning methods such as BPTT are essential for a powerful form of RL that is often referred to as Deep RL²⁶. There, one trains recurrent artificial neural networks with internally generated teaching signals. We show here that Deep RL can in principle also be carried out by neural networks of the brain, as e-prop approximates the performance of BPTT also in this RL context. However, another new ingredient is needed to prove that. Previous work on Deep RL for solving complex tasks, such as winning Atari games²⁶, required additional mechanisms to avoid well-known instabilities that arise from using nonlinear function approximators, such as the use of several interacting learners in parallel. As this parallel learning scheme does not appear to be biologically plausible, we introduce here a new method for avoiding learning instabilities: we show that a suitable schedule for the lengths of learning episodes and learning rates also alleviates learning instabilities in Deep RL.

We propose an online synaptic plasticity rule (5) for deep RL, which is similar to equation (3), except that a fading memory filter $\mathcal{F}_\gamma$ is applied here to the term $L_j^t \bar{e}_{ji}^t$, where γ is the given discount factor for future rewards and $\bar{e}_{ji}^t$ denotes a low-pass filtered copy of the eligibility trace e_{ji}^t (see Methods). This term is multiplied in the synaptic plasticity rule with the reward prediction error $\delta^t = r^t + \gamma V^{t+1} - V^t$, where r^t is the reward received at time t . This yields an instantaneous weight change of the form:

$$\Delta W_{ji}^t = -\eta \delta^t \mathcal{F}_\gamma(L_j^t \bar{e}_{ji}^t). \quad (5)$$

Previous three-factor learning rules for RL were usually of the form $\Delta W^t = \eta \delta^t \bar{e}_{ji}^{12,28}$. Hence, they estimated gradients of the policy just by correlating the output of network neurons with the reward prediction error. The learning power of this approach is known to be quite limited owing to high noise in the resulting gradient estimates. In contrast, in the plasticity rule (5) for reward-based e-prop, the eligibility traces are first combined with a neuron-specific feedback L_j^t , before they are multiplied with the

reward prediction error δ^t . We show in Methods analytically that this yields estimates of policy- and value gradients similarly as in deep RL with BPTT. Furthermore, in contrast to previously proposed three-factor learning rules, this rule (5) is also applicable to LSNNs.

We tested reward-based e-prop on a classical benchmark task²⁶ for learning intelligent behavior from rewards: winning Atari video games provided by the Arcade Learning Environment²⁹. To win such game, the agent needs to learn to extract salient information from the pixels of the game screen, and to infer the value of specific actions, even if rewards are obtained in a distant future. In fact, learning to win Atari games is a serious challenge for RL even in machine learning²⁶. Besides, artificial neural networks and BPTT, previous solutions also required experience replay (with a perfect memory of many frames and action sequences that occurred much earlier) or an asynchronous training of numerous parallel agents sharing synaptic weight updates. We show here that also an LSNN can learn via e-prop to win Atari games, through online learning of a single agent. This becomes possible with a single agent and without episode replay if the agent uses a schedule of increasing episode lengths—with a learning rate that is inversely related to that length. Using this scheme, an agent can experience diverse and uncorrelated short episodes in the first phase of learning, producing useful skills. Subsequently, the agent can fine-tune its policy using longer episodes.

First, we considered the well-known Atari game Pong (Fig. 4a). Here, the agent has to learn to hit a ball in a clever way using up and down movements of his paddle. A reward is obtained if the opponent cannot catch the ball. We trained an agent using reward-based e-prop for this task, and show a sample trial in Fig. 4c and Supplementary Movie 4. In contrast to common deep RL solutions, the agent learns here in a strict online manner, receiving at any time just the current frame of the game screen. In Fig. 4d, we demonstrate that also this biologically realistic learning approach leads to a competitive score.

If one does not insist on an online setting where the agent receives just the current frame of the video screen but the last four frames, winning strategies for about half of the Atari games can already be learnt by feedforward neural networks (see Supplementary Table 3 of ref. 26). However, for other Atari games, such as Fishing Derby (Fig. 5a), it was even shown in ref. 26 that deep RL applied to LSTM networks achieves a substantially higher score than any deep RL method for feedforward networks, which was considered there. Hence, in order to test the power of online reward-based e-prop also for those Atari games that require enhanced temporal processing, we tested it on the Fishing Derby game. In this game, the agent has to catch as many fish as possible while avoiding that the shark touches the fish with any part of its body, and that the opponent catches the fish first. We show in Fig. 5c that online reward-based e-prop applied to an LSNN does in fact reach the same performance as reference offline algorithms applied to LSTM networks. We show a random trial after learning in Fig. 5d, where we can identify two different learnt behaviors: first, by evading the shark, and a second by collecting fish. The agent has learnt to switch between these two behaviors as required by the situation.

In general, we conjecture that variants of reward-based e-prop will be able to solve most deep RL tasks that can be solved by online actor-critic methods in machine learning.

Discussion

We propose that in order to understand the computational function and neural coding of neural networks in the brain, one needs to understand the organization of the plasticity

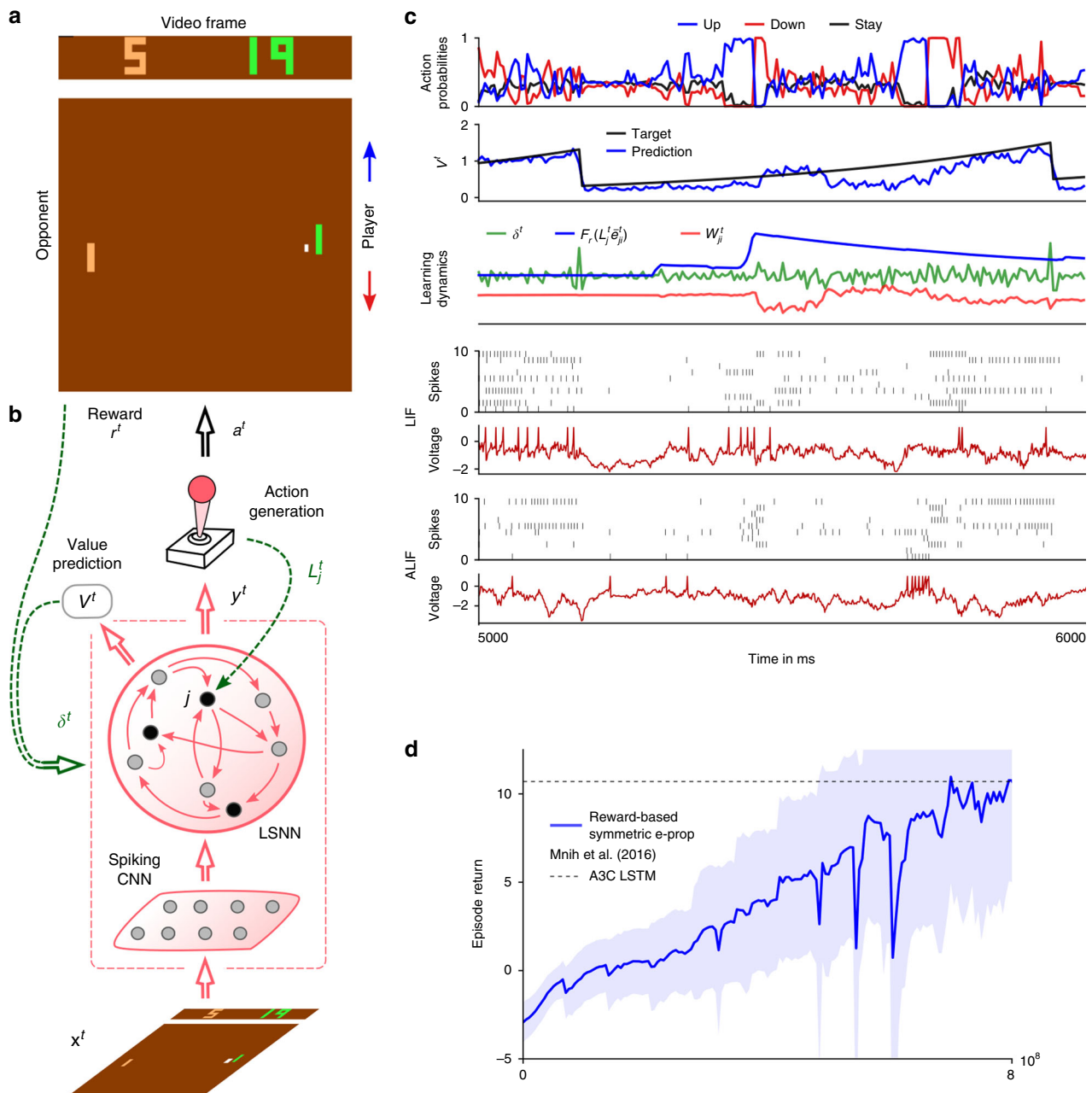


Fig. 4 Application of e-prop to the Atari game Pong. **a** Here, the player (green paddle) has to outplay the opponent (light brown). A reward is acquired when the opponent cannot bounce back the ball (a small white square). To achieve this, the agent has to learn to hit the ball also with the edges of his paddle, which causes a less predictable trajectory. **b** The agent is realized by an LSNN. The pixels of the current video frame of the game are provided as input. During processing of the stream of video frames by the LSNN, actions are generated by the stochastic policy in an online manner. At the same time, future rewards are predicted. The current error in prediction is fed back both to the LSNN and the spiking CNN that preprocesses the frames. **c** Sample trial of the LSNN after learning with reward-based e-prop. From top to bottom: probabilities of stochastic actions, prediction of future rewards, learning dynamics of a random synapse (arbitrary units), spiking activity of 10 out of 240 sample LIF neurons and 10 out of 160 sample ALIF neurons, and membrane potentials (more precisely: $v_j^t - A_j^t$) for the two sample neurons j at the bottom of the spike raster above. **d** Learning progress of the LSNN trained with reward-based e-prop, reported as the sum of collected rewards during an episode. The learning curve is averaged over five different runs and the shaded area represents the standard deviation. More information about the comparison between our results and A3C are given in Supplementary Note 5.

mechanisms that install and maintain these. So far, BPTT was the only candidate for that, as no other learning method provided sufficiently powerful computational function to RSNN models. But as BPTT is not viewed to be biologically realistic⁵, it does not help us to understand learning in the brain. E-prop offers a solution to this dilemma, as it does not require biologically unrealistic mechanisms, but still enables RSNNs to learn difficult

computational tasks, in fact almost as well as BPTT. Furthermore, it enables RSNNs to solve these tasks in an energy-efficient sparse firing regime, rather than resorting to rate coding.

E-prop relies on two types of signals that are abundantly available in the brain, but whose precise role for learning have not yet been understood: eligibility traces and learning signals. As e-prop is based on a transparent mathematical principle (see Eq.

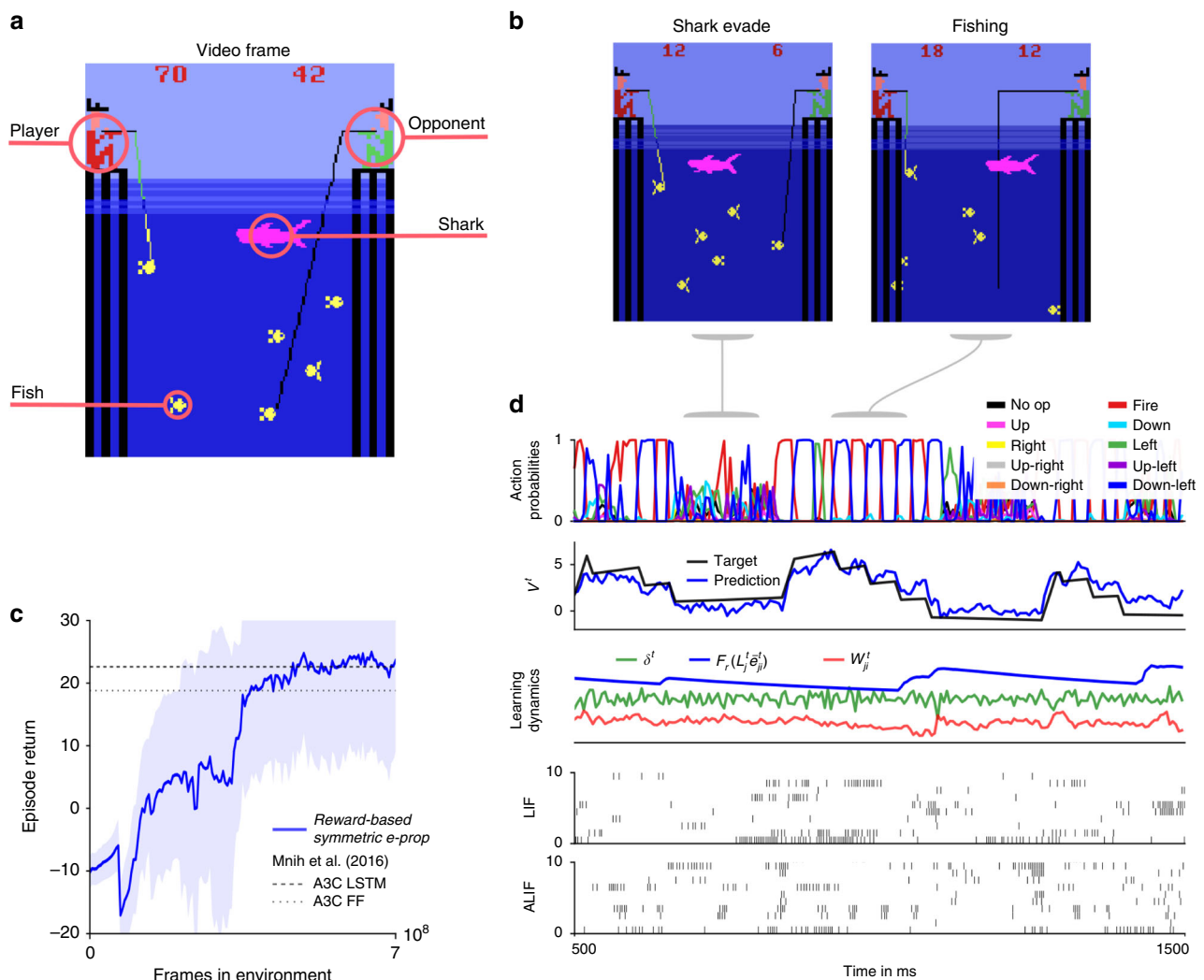


Fig. 5 Application of e-prop to learning to win the Atari game Fishing Derby. **a** Here the player has to compete against an opponent, and try to catch more fish from the sea. **b** Once a fish has bit, the agent has to avoid that the fish gets touched by a shark. **c** Sample trial of the trained network. From top to bottom: probabilities of stochastic actions, prediction of future rewards, learning dynamics of a random synapse (arbitrary units), spiking activity of 20 out of 180 sample LIF neurons and 20 out of 120 sample ALIF neurons. **d** Learning curves of an LSNN trained with reward-based e-prop as in Fig. 4d.

(3)), it provides a normative model for both types of signals, as well as for synaptic plasticity rules. Interestingly, the resulting learning model suggests that a characteristic aspect of many biological neurons—the presence of slowly changing hidden variables—provides a possible solution to the problem how a RSNN can learn without error signals that propagate backwards in time: slowly changing hidden variables of neurons cause eligibility traces that propagate forward over longer time spans, and are therefore able to coincide with later arising instantaneous error signals (see Fig. 3b).

The theory of e-prop makes a concrete experimentally testable prediction: that the time constant of the eligibility trace for a synapse is correlated with the time constant for the history-dependence of the firing activity of the postsynaptic neuron. It also suggests that the experimentally found diverse time constants of the firing activity of populations of neurons in different brain areas³⁰ are correlated with their capability to handle corresponding ranges of delays in temporal credit assignment for learning.

Finally, e-prop theory provides a hypothesis for the functional role of the experimentally found diversity of dopamine signals to different populations of neurons¹⁴. Whereas previous theories of reward-based learning required that the same learning signal is

sent to all neurons, the basic Eq. (1) for e-prop postulates that ideal top-down learning signals to a population of neurons depend on its impact on the network performance (loss function), and should therefore be target-specific (see Fig. 2c and Supplementary Note 6). In fact, the learning-to-learn result for e-prop in ref. ³¹ suggests that prior knowledge about the possible range of learning signals for a brain area could optimize top-down learning signals even further on an evolutionary time scale, thereby enabling for example learning from few or even a single trial.

Previous methods for training RSNNs did not aim at approximating BPTT. Instead, some of them were relying on control theory to train a chaotic reservoir of spiking neurons^{32–34}. Others used the FORCE algorithm^{35,36} or variants of it^{35,37–39}. However, the FORCE algorithm was not argued to be biologically realistic, as the plasticity rule for each synaptic weight requires knowledge of the current values of all other synaptic weights. The generic task considered in ref. ³⁵ was to learn with supervision how to generate patterns. We show in Supplementary Figs. 1 and 7 that RSNNs can learn such tasks also with a biologically plausible learning method e-prop.

Several methods for approximating stochastic gradient descent in feedforward networks of spiking neurons have been proposed,

see e.g., refs. 40–44. These employ—like e-prop—a pseudo-gradient to overcome the non-differentiability of a spiking neuron, as proposed previously in refs. 45,46. References 40,42,43 arrive at a synaptic plasticity rule for feedforward networks that consists—like e-prop—of the product of a learning signal and a derivative (eligibility trace) that describes the dependence of a spike of a neuron j on the weight of an afferent synapse W_{ji} . But in a recurrent network, the spike output of j depends on W_{ji} also indirectly, via loops in the network that allow that a spike of neuron j contributes to the firing of other neurons, which in turn affect the firing of the presynaptic neuron i . Hence, the corresponding eligibility trace can no longer be locally computed if one transfers these methods for feedforward networks to recurrently connected networks. Therefore, ref. 40 suggests the need to investigate extensions of their approach to RSNNs.

Previous work on the design of online gradient descent learning algorithms for non-spiking RNNs was based on real-time recurrent learning (RTRL)⁸. RTRL itself has rarely been used as its computational complexity per time step is $\mathcal{O}(n^4)$, if n is the number of neurons. But interesting approximations to RTRL have subsequently been proposed (see ref. 47 for a review): some stochastic approximations⁴⁸, which are $\mathcal{O}(n^3)$ or only applicable for small networks⁴⁹, and also recently two deterministic $\mathcal{O}(n^2)$ approximations^{50,51}. The latter were in fact written at the same time as the first publication of e-prop³¹. A structural difference between this paper and⁵⁰ is that their approach requires that learning signals are transmitted between the neurons in the RNN, with separately learnt weights⁵¹ derived for rate based neurons a learning rule similar to random e-prop. But this work did not address other forms of learning than supervised regression, such as RL, nor learning in networks of spiking neurons, or in more powerful types of RNNs with slow hidden variables such as LSTM networks or LSNNs.

E-prop also has complexity $\mathcal{O}(n^2)$, in fact $\mathcal{O}(S)$ if S is the number of synaptic connections. This bound is optimal—except for the constant factor—since this is the asymptotic complexity of just simulating the RNN. The key point of e-prop is that the general form (13) of its eligibility trace collects all contributions to the loss gradient that can be locally computed in a feedforward manner. This general form enables applications to spiking neurons with slowly varying hidden variables, such as neurons with firing rate adaptation, which are essential ingredients of RSNNs to reach the computational power of LSTM networks³. We believe that this approach can be extended in future work with a suitable choice of pseudo derivatives to a wide range of biologically more realistic neuron models. It also enables the combination of these rigorously derived eligibility traces with—semantically identical but algorithmically very different—eligibility traces from RL for reward-based e-prop (Eq. (5)), thereby bringing the power of deep RL to RSNNs. As a result, we were able to show in Figs. 2–5 that RSNNs can learn with the biologically plausible rules for synaptic plasticity that arise from the e-prop theory to solve tasks such as phoneme recognition, integrating evidence over time and waiting for the right moment to act, and winning Atari games. These are tasks that are fundamental for modern learning-based AI, but have so far not been solved with RSNNs. Hence, e-prop provides a new perspective of the major open question how intelligent behavior can be learnt and controlled by neural networks of the brain.

Apart from obvious consequences of e-prop for research in neuroscience and cognitive science, e-prop also provides an interesting new tool for approaches in machine learning where BPTT is replaced by approximations in order to improve computational efficiency. We have already shown in Supplementary Fig. 4 that e-prop provides a powerful online learning method for LSTM networks. Furthermore, the combination of eligibility

traces from e-prop with synthetic gradients from ref. 52 even improves performance of LSTM networks for difficult machine learning problems such as the copy-repeat task and the Penn Treebank word prediction task³¹. Other future extensions of e-prop could explore a combination with attention-based models in order to cover multiple timescales.

Finally, e-prop suggests a promising new approach for realizing powerful on-chip learning of RSNNs on neuromorphic chips. Whereas, BPTT is not within reach of current neuromorphic hardware, an implementation of e-prop appears to offer no serious hurdle. Our results show that an implementation of e-prop will provide a qualitative jump in on-chip learning capabilities of neuromorphic hardware.

Methods

Network models. To exhibit the generality of the e-prop approach, we define the dynamics of recurrent neural networks using a general formalism that is applicable to many recurrent neural network models, not only to RSNNs and LSNNs. Also non-spiking models such as LSTM networks fit under this formalism (see Supplementary Note 4). The network dynamics is summarized by the computational graph in Fig. 6. Denoting the observable states (e.g., spikes) as z^t , the hidden states as h^t , the inputs as x^t and using W_j to gather all weights of synapses arriving at neuron j , the function M defines the update of the hidden state of a neuron j : $h_j^t = M(h_j^{t-1}, z^{t-1}, x^t, W_j)$ and the function f defines the update of the observable state of a neuron j : $z_j^t = f(h_j^t, z^{t-1}, x^t, W_j)$ (f simplifies to $z_j^t = f(h_j^t)$ for LIF and ALIF neurons). We chose a discrete time step of $\delta t = 1$ ms for all our simulations. Control experiments with smaller time steps for the task of Fig. 3, reported in Supplementary Fig. 6, suggest that the size of the time step has no significant impact on the performance of e-prop.

LIF neurons. Each LIF neuron has a one-dimensional internal state—or hidden variable— h_j^t that consists only of the membrane potential v_j^t . The observable state $z_j^t \in \{0, 1\}$ is binary, indicating a spike ($z_j^t = 1$) or no spike ($z_j^t = 0$) at time t . The dynamics of the LIF model is defined by the equations:

$$v_j^{t+1} = \alpha v_j^t + \sum_{i \neq j} W_{ji}^{\text{rec}} z_i^t + \sum_i W_{ji}^{\text{in}} x_i^{t+1} - z_j^t v_{\text{th}} \quad (6)$$

$$z_j^t = H(v_j^t - v_{\text{th}}). \quad (7)$$

W_{ji}^{rec} (W_{ji}^{in}) is the synaptic weight from network (input) neuron i to neuron j . The decay factor α in (6) is given by $e^{-\delta t/\tau_m}$, where τ_m (typically 20 ms) is the membrane time constant. δt denotes the discrete time step size, which is set to 1 ms in our simulations. H denotes the Heaviside step function. Note that we deleted in Eq. (6) the factor $1 - \alpha$ that occurred in the corresponding equation (4) in the supplement of ref. 3. This simplifies the notation in our derivations, and has no impact on the model if parameters like the threshold voltage are scaled accordingly.

Owing to the term $-z_j^t v_{\text{th}}$ in Eq. (6), the neurons membrane potential is reduced by a constant value after an output spike, which relates our model to the spike response model⁵³. To introduce a simple model of neuronal refractoriness, we further assume that z_j^t is fixed to 0 after each spike of neuron j for a short refractory period of 2–5 ms depending on the simulation.

LSNNs. According to the database of the Allen Institute², a fraction of neurons between ~20% (in mouse visual cortex) and 40% (in the human frontal lobe) exhibit SFA. It had been shown in ref. 3 that the inclusion of neuron models with SFA—via a time-varying firing threshold as slow hidden variable—drastically enhances computing capabilities of RSNN models. Hence, we consider here the same simple model for neurons with SFA as in ref. 3, to which we refer as ALIF neuron. This model is basically the same as the GLIF₂ model in the Technical White Paper on generalized LIF (GLIF) models from ref. 2. LSNNs are recurrently connected networks that consist of LIF and ALIF neurons. ALIF neurons j have a second hidden variable a_j^t , which denotes the variable component of its firing threshold. As a result, their internal state is a two-dimensional vector $h_j^t \stackrel{\text{def}}{=} [v_j^t, a_j^t]$. Their threshold potential A_j^t increases with every output spike and decreases exponentially back to the baseline threshold v_{th} . This can be described by

$$A_j^t = v_{\text{th}} + \beta a_j^t, \quad (8)$$

$$z_j^t = H(v_j^t - A_j^t), \quad (9)$$

with a threshold adaptation according to

$$a_j^{t+1} = \rho a_j^t + z_j^t, \quad (10)$$

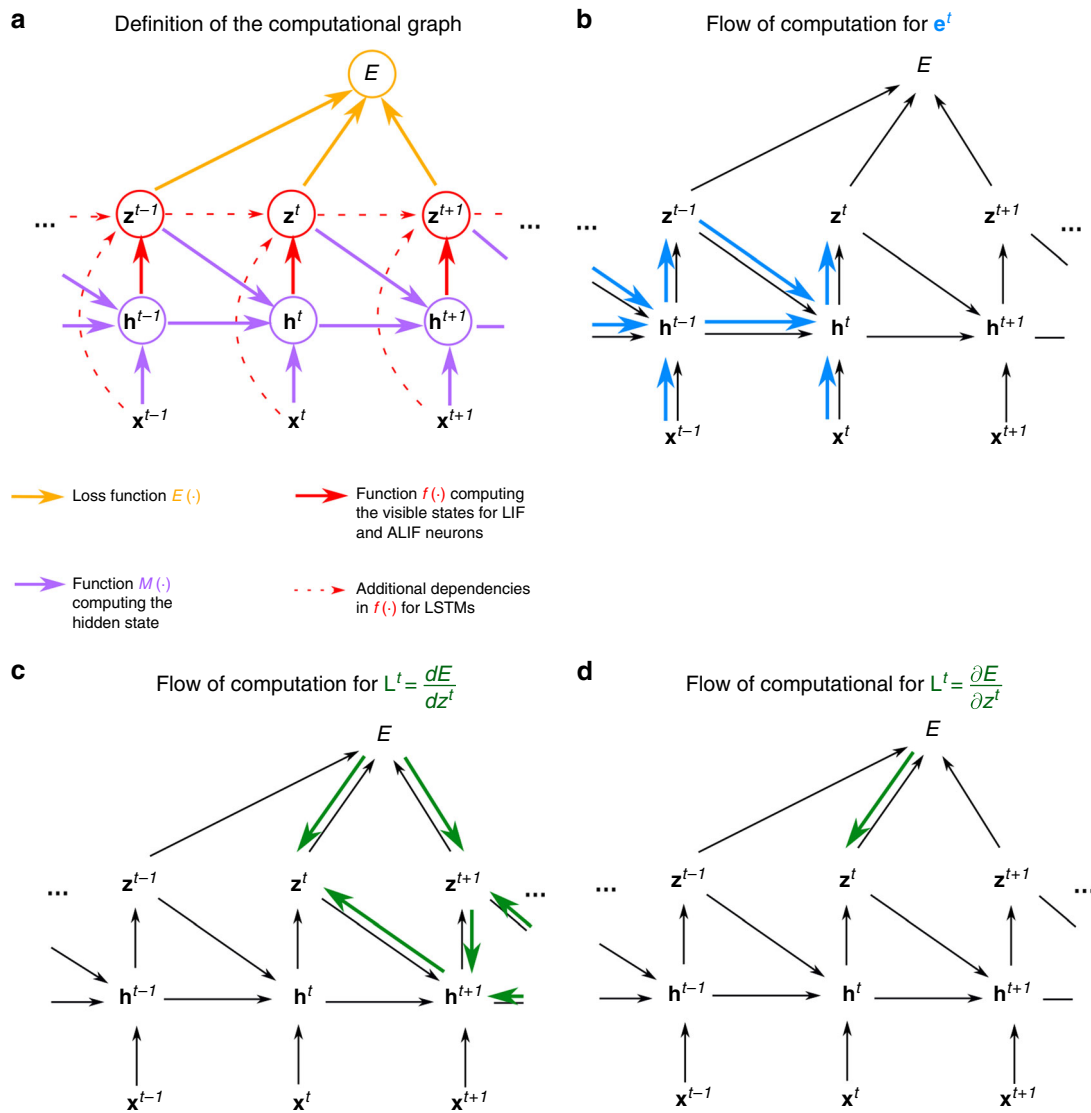


Fig. 6 Computational graph and gradient propagations. **a** Assumed mathematical dependencies between hidden neuron states $\mathbf{h}_j^t$, neuron outputs $\mathbf{z}^t$, network inputs $\mathbf{x}^t$, and the loss function E through the mathematical functions $E(\cdot)$, $M(\cdot)$, $f(\cdot)$ are represented by colored arrows. **b–d** The flow of computation for the two components $\mathbf{e}^t$ and $\mathbf{L}^t$ that merge into the loss gradients of Eq. (3) can be represented in similar graphs. **b** Following Eq. (14), the flow of the computation of the eligibility traces $\mathbf{e}_{ij}^t$ is going forward in time. **c** Instead, the ideal learning signals $\mathbf{L}_j^t = \frac{dE}{dz^t}$ requires to propagate gradients backward in time. **d** Hence, while $\mathbf{e}_{ij}^t$ is computed exactly, $\mathbf{L}_j^t$ is approximated in e-prop applications to yield an online learning algorithm.

where the decay factor ρ is given by $e^{-\delta t/\tau_a}$, and τ_a is the adaptation time constant that is typically chosen to be on the same time scale as the length of the working memory that is a relevant for a given task. The effects of SFA can last for several seconds in neocortical neurons, in factor up to 20 s according to experimental data⁵⁴. We refer to a RSNN as LSNN if some of its neurons are adaptive. We chose a fraction between 25 and 40% of the neurons to be adapting.

In relation to the more general formalism represented in the computational graph in Fig. 6, Eqs. (6) and (10) define $M(\mathbf{h}_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$, and Eqs. (7) and (9) define $f(\mathbf{h}_j^t)$.

Gradient descent for RSNNs. Gradient descent is problematic for spiking neurons because of the step function H in Eq. (7). We overcome this issue as in refs. 3,55; the non-existing derivative $\frac{\partial z_j^t}{\partial v_j^t}$ is replaced in simulations by a simple nonlinear function of the membrane potential that is called the pseudo derivative. Outside of the refractory period, we choose a pseudo derivative of the form $\psi_j^t =$

$\frac{1}{v_{th}} \gamma_{pd} \max\left(0, 1 - \frac{|v_j^t - A_j^t|}{v_{th}}\right)$ where $\gamma_{pd} = 0.3$ for ALIF neurons, and for LIF neurons A_j^t is replaced by v_{th} . During the refractory period the pseudo derivative is set to 0.

Network output and loss functions. We assume that network outputs y_k^t are real-valued and produced by leaky output neurons (readouts) k , which are not

recurrently connected:

$$y_k^t = \kappa y_k^{t-1} + \sum_j W_{kj}^{\text{out}} z_j^t + b_k^{\text{out}}, \quad (11)$$

where $\kappa \in [0, 1]$ defines the leak and b_k^{out} denotes the output bias. The leak factor κ is given for spiking neurons by $e^{-\delta t/\tau_{\text{out}}}$, where τ_{out} is the membrane time constant. Note that for non-spiking neural networks (such as for LSTM networks), temporal smoothing of the network observable state is not necessary. In this case, one can use $\kappa = 0$.

The loss function $E(\mathbf{z}^1, \dots, \mathbf{z}^T)$ quantifies the network performance. We assume that it depends only on the observable states $\mathbf{z}^1, \dots, \mathbf{z}^T$ of the network neurons. For instance, for a regression problem we define E as the mean square error $E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2$ between the network outputs y_k^t and target values $y_k^{*,t}$. For classification or RL tasks the loss function, E has to be re-defined accordingly.

Notation for derivatives. We distinguish the total derivative $\frac{dE}{dz^t}(\mathbf{z}^1, \dots, \mathbf{z}^T)$, which takes into account how E depends on $\mathbf{z}^t$ also indirectly through influence of $\mathbf{z}^t$ on the other variables $\mathbf{z}^{t+1}, \dots, \mathbf{z}^T$, and the partial derivative $\frac{\partial E}{\partial z^t}(\mathbf{z}^1, \dots, \mathbf{z}^T)$, which quantifies only the direct dependence of E on $\mathbf{z}^t$.

Analogously for the hidden state $\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$, the partial derivative $\frac{\partial M}{\partial \mathbf{h}_j^{t-1}}$ denotes the partial derivative of M with respect to $\mathbf{h}_j^{t-1}$. It only

quantifies the direct influence of $\mathbf{h}_j^{t-1}$ on $\mathbf{h}_j^t$ and it does not take into account how $\mathbf{h}_j^{t-1}$ indirectly influences $\mathbf{h}_j^t$ via the observable states $\mathbf{z}^{t-1}$. To improve readability, we also use the following abbreviations: $\frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \stackrel{\text{def}}{=} \frac{\partial M}{\partial \mathbf{h}_j^{t-1}}(\mathbf{h}_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$, $\frac{\partial \mathbf{h}_j^t}{\partial \mathbf{W}_j} \stackrel{\text{def}}{=} \frac{\partial M}{\partial \mathbf{W}_j}(\mathbf{h}_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$, and $\frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \stackrel{\text{def}}{=} \frac{\partial f}{\partial \mathbf{h}_j^t}(\mathbf{h}_j^t, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$.

Notation for temporal filters. For ease of notation, we use the operator $\mathcal{F}_\alpha$ to denote the low-pass filter such that, for any time series x_t :

$$\mathcal{F}_\alpha(x^t) = \alpha \mathcal{F}_\alpha(x^{t-1}) + x^t, \quad (12)$$

and $\mathcal{F}_\alpha(x^0) = x^0$. In the specific case of the time series z_j^t and e_{ji}^t , we simplify notation further and write $\bar{z}_j^t$ and $\bar{e}_{ji}^t$ for $\mathcal{F}_\alpha(z_j^t)$ and $\mathcal{F}_\alpha(e_{ji}^t)$.

Mathematical basis for e-prop. We provide here the proof of the fundamental Eq. (1) for e-prop (for the case where the learning signal L_j^t has its ideal value $\frac{dE}{dz_j^t}$)

$$\frac{dE}{dW_{ji}} = \sum_t \frac{dE}{dz_j^t} \cdot \left[\frac{dz_j^t}{dW_{ji}} \right]_{\text{local}},$$

with the eligibility trace

$$e_{ji}^t \stackrel{\text{def}}{=} \left[\frac{dz_j^t}{dW_{ji}} \right]_{\text{local}} \stackrel{\text{def}}{=} \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \underbrace{\sum_{t' \leq t} \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}}_{\stackrel{\text{def}}{=} e_{ji}^{t'}}. \quad (13)$$

The second factor e_{ji}^t , which we call eligibility vector, obviously satisfies the recursive equation

$$\epsilon_{ji}^t = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdot \epsilon_{ji}^{t-1} + \frac{\partial \mathbf{h}_j^t}{\partial W_{ji}}, \quad (14)$$

where $\cdot$ denotes the dot product. This provides the rule for the online computation of ϵ_{ji}^t , and hence of $e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \epsilon_{ji}^t$.

We start from a classical factorization of the loss gradients in recurrent neural networks that arises for instance in equation (12) of ref. ⁵⁶ to describe BPTT. This classical factorization can be justified by unrolling an RNN into a large feedforward network, where each layer (l) represents one time step. In a feedforward network, the loss gradients with respect to the weights $W_{ji}^{(l)}$ of layer (l) are given by

$\frac{dE}{dW_{ji}^{(l)}} = \frac{dE}{dh_j^{(l)}} \frac{\partial h_j^{(l)}}{\partial W_{ji}^{(l)}}$. But as the weights are shared across the layers when representing a recurrent network, the summation of these gradients over the layers l of the unrolled RNN yields this classical factorization of the loss gradients:

$$\frac{dE}{dW_{ji}} = \sum_{t'} \frac{dE}{dh_j^{t'}} \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}. \quad (15)$$

Note that the first factor $\frac{dE}{dh_j^{t'}}$ in these products also needs to take into account how the internal state $\mathbf{h}_j$ of neuron j evolves during subsequent time steps, and whether it influences firing of j at later time steps. This is especially relevant for ALIF neurons and other biologically realistic neuron models with slowly changing internal states. Note that this first factor of (15) is replaced in the e-prop equation (13) by the derivative $\frac{dE}{dz_j^t}$ of E with regard to the observable variable z_j^t . There, the evolution of the internal state of neuron j is pushed into the second factor, the eligibility trace e_{ji} , which collects in e-prop all online computable factors of the loss gradient that just involve neurons j and i .

Now we show that one can re-factorize the expression (15) and prove that the loss gradients can also be computed using the new factorization (13) that underlies e-prop. In the steps of the subsequent proof until Eq. (19), we decompose the term $\frac{dE}{dh_j^{t'}}$ into a series of learning signals $L_j^t = \frac{dE}{dz_j^t}$ and local factors $\frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}}$ for $t \geq t'$. Those local factors will later be used to transform the partial derivative $\frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}$ from Eq. (15) into the eligibility vector ϵ_{ji}^t that integrates the whole history of the synapse up to time t , not just a single time step. To do so, we express $\frac{dE}{dh_j^{t'+1}}$ recursively as a function of the same derivative at the next time step $\frac{dE}{dh_j^{t'+2}}$ by applying the chain rule at the node $\mathbf{h}_j^t$ for $t = t'$ of the computational graph shown in Fig. 6c:

$$\frac{dE}{dh_j^{t'}} = \frac{dE}{dz_j^{t'}} \frac{\partial z_j^{t'}}{\partial \mathbf{h}_j^{t'}} + \frac{dE}{d\mathbf{h}_j^{t'+1}} \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \quad (16)$$

$$= L_j^{t'} \frac{\partial z_j^{t'}}{\partial \mathbf{h}_j^{t'}} + \frac{dE}{d\mathbf{h}_j^{t'+1}} \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}}, \quad (17)$$

where we defined the learning signal $L_j^{t'}$ as $\frac{dE}{dz_j^{t'}}$. The resulting recursive expansion

ends at the last time step T of the computation of the RNN, i.e., $\frac{dE}{dh_j^{T+1}} = 0$. If one repeatedly substitutes the recursive formula (17) into the classical factorization (15) of the loss gradients, one gets:

$$\frac{dE}{dW_{ji}} = \sum_{t'} \left(L_j^{t'} \frac{\partial z_j^{t'}}{\partial \mathbf{h}_j^{t'}} + \frac{dE}{d\mathbf{h}_j^{t'+1}} \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \right) \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}} \quad (18)$$

$$= \sum_{t'} \left(L_j^{t'} \frac{\partial z_j^{t'}}{\partial \mathbf{h}_j^{t'}} + \left(L_j^{t'+1} \frac{\partial z_j^{t'+1}}{\partial \mathbf{h}_j^{t'+1}} + (\cdots) \frac{\partial \mathbf{h}_j^{t'+2}}{\partial \mathbf{h}_j^{t'+1}} \right) \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \right) \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}. \quad (19)$$

The following equation is the main equation for understanding the transformation from BPTT into e-prop. The key idea is to collect all terms $\frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}}$, which are

multiplied with the learning signal L_j^t at a given time t . These are only terms that concern events in the computation of neuron j up to time t , and they do not depend on other future losses or variable values. To this end, we write the term in parentheses in Eq. (19) into a second sum indexed by t and exchange the summation indices to pull out the learning signal L_j^t . This expresses the loss gradient of E as a sum of learning signals L_j^t multiplied by some factor indexed by ji , which we define as the eligibility trace $e_{ji}^t \in \mathbb{R}$. The main factor of it is the eligibility vector $\epsilon_{ji}^t \in \mathbb{R}^d$, which has the same dimension as the hidden state $\mathbf{h}_j^t$:

$$\frac{dE}{dW_{ji}} = \sum_{t'} \sum_{t \geq t'} L_j^t \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}} \quad (20)$$

$$= \sum_t L_j^t \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \underbrace{\sum_{t' \leq t} \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}}}_{\stackrel{\text{def}}{=} e_{ji}^t} \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}. \quad (21)$$

This completes the proof of Eqs. (1), (3), (13).

Derivation of eligibility traces LIF neurons. The eligibility traces for LSTMs are derived in the supplementary materials. Below we provide the derivation of eligibility traces for spiking neurons.

We compute the eligibility trace of a synapse of a LIF neuron without adaptive threshold (Eq. (6)). Here, the hidden state $\mathbf{h}_j^t$ of a neuron consists just of the membrane potential v_j^t and we have $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} = \frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$ and $\frac{\partial v_j^t}{\partial W_{ji}} = z_i^{t-1}$ (for a derivation of the eligibility traces taking the reset into account we refer to Supplementary Note 1). Using these derivatives and Eq. (14), one obtains that the eligibility vector is the low-pass filtered presynaptic spike-train,

$$\epsilon_{ji}^{t+1} = \mathcal{F}_\alpha(z_i^t) \stackrel{\text{def}}{=} \bar{z}_i^t, \quad (22)$$

and following Eq. (13), the eligibility trace is:

$$e_{ji}^{t+1} = \psi_j^{t+1} \bar{z}_i^t. \quad (23)$$

For all neurons, j the derivations in the next sections also hold for synaptic connections from input neurons i , but one needs to replace the network spikes z_i^{t-1} by the input spikes x_i^t (the time index switches from $t-1$ to t because the hidden state $\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$ is defined as a function of the input at time t but the preceding recurrent activity). For simplicity, we have focused on the case where transmission delays between neurons in the RSNN are just 1 ms. If one uses more realistic length of delays d , this d appears in Eqs. (23)–(25) instead of -1 as the most relevant time point for presynaptic firing (see Supplementary Note 1). This moves resulting synaptic plasticity rules closer to experimentally observed forms of STDP.

Eligibility traces for ALIF neurons. The hidden state of an ALIF neuron is a two dimensional vector $\mathbf{h}_j^t = [v_j^t, a_j^t]$. Hence, a two-dimensional eligibility vector

$\epsilon_{ji}^t \stackrel{\text{def}}{=} [\epsilon_{ji,v}^t, \epsilon_{ji,a}^t]$ is associated with the synapse from neuron i to neuron j , and the matrix $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ is a 2×2 matrix. The derivatives $\frac{\partial a_j^{t+1}}{\partial a_j^t}$ and $\frac{\partial v_j^{t+1}}{\partial v_j^t}$ capture the dynamics of the adaptive threshold. Hence, to derive the computation of eligibility traces, we substitute the spike z_j in Eq. (10) by its definition given in Eq. (9). With this convention, one finds that the diagonal of the matrix $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ is formed by the terms $\frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$ and $\frac{\partial a_j^{t+1}}{\partial a_j^t} = \rho - \psi_j^t \beta$. Above and below the diagonal, one finds, respectively, $\frac{\partial v_j^{t+1}}{\partial a_j^t} = 0$, $\frac{\partial a_j^{t+1}}{\partial v_j^t} = \psi_j^t$. Seeing that $\frac{\partial \mathbf{h}_j^t}{\partial W_{ji}} = \left[\frac{\partial v_j^t}{\partial W_{ji}}, \frac{\partial a_j^t}{\partial W_{ji}} \right] = [z_i^{t-1}, 0]$, one can finally compute the eligibility traces using Eq. (13). The component of the eligibility vector associated with the membrane potential remains the same as in the LIF case and only depends on the presynaptic neuron: $\epsilon_{ji,v}^t = \bar{z}_i^{t-1}$. For the component

associated with the adaptive threshold we find the following recursive update:

$$e_{ji,a}^{t+1} = \psi_j^t \bar{z}_i^{t-1} + (\rho - \psi_j^t \beta) e_{ji,a}^t, \quad (24)$$

and, since $\frac{\partial z_j^t}{\partial \mathbf{y}^t} = \left[\frac{\partial z_j^t}{\partial \mathbf{y}^t}, \frac{\partial z_j^t}{\partial \mathbf{y}^t} \right] = [\psi_j^t, -\beta \psi_j^t]$, this results in an eligibility trace of the form:

$$e_{ji}^t = \psi_j^t (\bar{z}_i^{t-1} - \beta e_{ji,a}^t). \quad (25)$$

Recall that the constant $\rho = \exp(-\frac{\delta t}{\tau_a})$ arises from the adaptation time constant τ_a , which typically lies in the range of hundreds of milliseconds to a few seconds in our experiments, yielding values of ρ between 0.995 and 0.9995. The constant β is typically of the order of 0.07 in our experiments.

To provide a more interpretable form of eligibility trace that fits into the standard form of local terms considered in three-factor learning rules¹², one may drop the term $-\psi_j^t \beta$ in Eq. (24). This approximation $\tilde{e}_{ji,a}^t$ of Eq. (24) becomes an exponential trace of the post-pre pairings accumulated within a time window as large as the adaptation time constant:

$$\tilde{e}_{ji,a}^{t+1} = \mathcal{F}_\rho(\psi_j^t \bar{z}_i^{t-1}). \quad (26)$$

The eligibility traces are computed with Eq. (24) in most experiments, but the performances obtained with symmetric e-prop and this simplification were indistinguishable in the task where temporal credit assignment is difficult of Fig. 3.

Synaptic plasticity rules resulting from e-prop. An exact computation of the ideal learning signal $\frac{dE}{dz_j}$ in Eq. (1) requires to back-propagate gradients through time (see Fig. 6c). For online e-prop, we replace it with the partial derivative $\frac{\partial E}{\partial z_j}$, which can be computed online. Implementing the weight updates with gradient descent and learning rate η , all the following plasticity rules are derived from the formula

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \frac{\partial E}{\partial z_j} e_{ji}^t. \quad (27)$$

Note that in the absence of the superscript t , ΔW_{ji} denotes the cumulated weight change over one trial or batch of consecutive trials but not the instantaneous weight update. This can be implemented online by accumulating weight updates in a hidden synaptic variable. Note also that the weight updates derived in the following for the recurrent weights W_{ji}^{rec} also apply to the inputs weights W_{ji}^{in} . For the output weights and biases, the derivation does not require the theory of e-prop, and the weight updates can be found in Supplementary Note 3.

In the case of a regression problem with targets $y_k^{*,t}$ and outputs y_k^t defined in Eq. (11), we define the loss function $E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2$. This results in a partial derivative of the form $\frac{\partial E}{\partial z_j^t} = \sum_k W_{kj}^{\text{out}} \sum_{t' \geq t} (y_k^{t'} - y_k^{*,t'}) \kappa^{t'-t}$. This seemingly provides an obstacle for online learning, because the partial derivative is a weighted sum over future errors. But this problem can be resolved as one can interchange the two summation indices in the expression for the weight updates (see Supplementary Note 3). In this way, the sum over future events transforms into a low-pass filtering of the eligibility traces $\tilde{e}_{ji}^t = \mathcal{F}_\kappa(e_{ji}^t)$, and the resulting weight update can be written as

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \underbrace{\left(\sum_k B_{jk} (y_k^t - y_k^{*,t}) \right)}_{=L_j^t} \tilde{e}_{ji}^t. \quad (28)$$

For classification tasks, we assume that K target categories are provided in the form of a one-hot encoded vector $\pi^{*,t}$ with K dimensions. We define the probability for class k predicted by the network as $\pi_k^t = \text{softmax}_k(y_1^t, \dots, y_K^t) = \exp(y_k^t) / \sum_{k'} \exp(y_{k'}^t)$, and the loss function for classification tasks as the cross-entropy error $E = -\sum_{t,k} \pi_k^{*,t} \log \pi_k^t$. The plasticity rule resulting from e-prop reads (see derivation in Supplementary Note 3):

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \underbrace{\left(\sum_k B_{jk} (\pi_k^t - \pi_k^{*,t}) \right)}_{=L_j^t} \tilde{e}_{ji}^t. \quad (29)$$

Reward-based e-prop: application of e-prop to deep RL. For RL, the network interacts with an external environment. At any time, t the environment can provide a positive or negative reward r^t . Based on the observations $\mathbf{x}^t$ that are perceived, the network has to commit to actions $a^0, \dots, a^n, \dots$ at certain decision times $t_0, \dots, t_n, \dots$. Each action a^t is sampled from a probability distribution $\pi(\cdot | \mathbf{y}^t)$, which is also referred to as the policy of the RL agent. The policy is defined as function of the network outputs $\mathbf{y}^t$, and is chosen here to be a categorical distribution of K discrete action choices. We assume that the agent chooses action k with probability $\pi_k^t = \pi(a^t = k | \mathbf{y}^t) = \text{softmax}_k(y_1^t, \dots, y_K^t) = \exp(y_k^t) / \sum_{k'} \exp(y_{k'}^t)$.

The goal of RL is to maximize the expected sum of discounted rewards. That is, we want to maximize the expected return at time $t=0$, $\mathbb{E}[R^0]$, where the return at

time t is defined as $R^t = \sum_{t' \geq t} \gamma^{t'-t} r^{t'}$ with a discount factor $\gamma \leq 1$. The expectation is taken over the agent actions a^t , the rewards r^t and the observations from the environment $\mathbf{x}^t$. We approach this optimization problem by using the actor-critic variant of the policy gradient algorithm, which applies gradient ascent to maximize $\mathbb{E}[R^0]$. The basis of the estimated gradient relies on an estimation of the policy gradient, as shown in Section 13.3 in ref. 27. There, the resulting weight update is given in equation (13.8) where G_t refers to the return R^t . Similarly, the gradient $\frac{d\mathbb{E}[R^0]}{dW_{ji}}$ is proportional to $\mathbb{E} \left[\sum_{t_n} R^{t_n} \frac{d \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{dW_{ji}} \right]$, which is easier to compute because the expectation can be estimated by an average over one or many trials. Following this strategy, we define the per-trial loss function E_π as a function of the sequence of actions $a^0, \dots, a^n, \dots$ and rewards $r^0, \dots, r^T$ sampled during this trial:

$$E_\pi(a^0, \dots, a^n, r^0, \dots, r^T) \stackrel{\text{def}}{=} - \sum_n R^{t_n} \log \pi(a^{t_n} | \mathbf{y}^{t_n}). \quad (30)$$

And thus:

$$\frac{d\mathbb{E}[R^0]}{dW_{ji}} \propto \mathbb{E} \left[\sum_{t_n} R^{t_n} \frac{d \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{dW_{ji}} \right] = -\mathbb{E} \left[\frac{dE_\pi}{dW_{ji}} \right]. \quad (31)$$

Intuitively, given a trial with high rewards, policy gradient changes the network output $\mathbf{y}$ to increase the probability of the actions a^n that occurred during this trial. In practice, the gradient $\frac{dE_\pi}{dW_{ji}}$ is known to have high variance and the efficiency of the learning algorithm can be improved using the actor-critic variant of the policy gradient algorithm. It involves the policy π (the actor) and an additional output neuron V , which predicts the value function $\mathbb{E}[R^t]$ (the critic). The actor and the critic are learnt simultaneously by defining the loss function as

$$E = E_\pi + c_V E_V, \quad (32)$$

where $E_\pi = -\sum_n R^{t_n} \log \pi(a^{t_n} | \mathbf{y}^{t_n})$ measures the performance of the stochastic policy π , and $E_V = \sum_t \frac{1}{2} (R^t - V^t)^2$ measures the accuracy of the value estimate V^t .

As V^t is independent of the action a^t one can show that $0 = \mathbb{E} \left[V^{t_n} \frac{d \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{dW_{ji}} \right]$.

We can use that to define an estimator $\widehat{\frac{dE}{dW_{ji}}}$ of the loss gradient with reduced variance:

$$\mathbb{E} \left[\frac{dE}{dW_{ji}} \right] = \mathbb{E} \left[\frac{dE_\pi}{dW_{ji}} \right] + c_V \mathbb{E} \left[\frac{dE_V}{dW_{ji}} \right] \quad (33)$$

$$= \mathbb{E} \left[\underbrace{- \sum_{t_n} (R^{t_n} - V^{t_n}) \frac{d \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{dW_{ji}} + c_V \frac{dE_V}{dW_{ji}}}_{\stackrel{\text{def}}{=} \widehat{\frac{dE}{dW_{ji}}}} \right], \quad (34)$$

similarly as in equation (13.11) of Section 13.4 in ref. 27. A difference in notation is that $b(S_t)$ refers to our value estimation V^t . In addition, Eq. (34) already includes the gradient $\frac{dE_V}{dW_{ji}}$ that is responsible for learning the value prediction. Until now, this derivation follows the classical definition of the actor-critic variant of policy gradient, and the gradient $\frac{dE}{dW_{ji}}$ can be computed with BPTT. To derive reward-based e-prop, we follow instead the generic online approximation of e-prop as in Eq. (27) and approximate $\widehat{\frac{dE}{dW_{ji}}}$ by a sum of terms of the form $\widehat{\frac{\partial E}{\partial z_j^t}} e_{ji}^t$ with

$$\widehat{\frac{\partial E}{\partial z_j^t}} = - \sum_n (R^{t_n} - V^{t_n}) \frac{\partial \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{\partial z_j^t} + c_V \frac{\partial E_V}{\partial z_j^t}. \quad (35)$$

We choose this estimator $\widehat{\frac{\partial E}{\partial z_j^t}}$ of the loss derivative because it is unbiased and has a low variance, more details are given in Supplementary Note 5. We derive below the resulting synaptic plasticity rule as needed to solve the task of Fig. 4, 5. For the case of a single action as used in Supplementary Fig. 5 we refer to Supplementary Note 5.

When there is a delay between the action and the reward or, even harder, when a sequence of many actions lead together to a delayed reward, the loss function E cannot be computed online because the evaluation of R^{t_n} requires knowledge of future rewards. To overcome this, we introduce temporal difference errors $\delta^t = r^t + \gamma V^{t+1} - V^t$ (see Fig. 4), and use the equivalence between the forward and backward view in RL²⁷. Using the one-hot encoded action $\mathbb{1}_{a^t=k}$ at time t , which assumes the value 1 if and only if $a^t = k$ (else it has value 0), we arrive at the following synaptic plasticity rules for a general actor-critic algorithm with e-prop (see Supplementary Note 5):

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \delta^t \mathcal{F}_{\mathcal{Y}} \left(L_j^t \tilde{e}_{ji}^t \right) \quad \text{for} \quad (36)$$

$$L_j^t = -c_V B_j^V + \sum_k B_{jk}^a (\pi_k^t - \mathbb{1}_{a^t=k}), \quad (37)$$

where we define the term $\pi_k^t - \mathbb{1}_{a^t=k}$ to have value zero when no action is taken at

time t . B_j^V is here the weight from the output neuron for the value function to neuron j , and the weights B_{jk}^{π} denote the weights from the outputs for the policy.

A combination of reward prediction error and neuron-specific learning signal was previously used in a plasticity rule for feedforward networks inspired by neuroscience^{57,58}. Here, it arises from the approximation of BPTT by e-prop in RSNNs solving RL problems. Note that the filtering $\mathcal{F}_y$ requires an additional eligibility trace per synapse. This arises from the temporal difference learning in RL²⁷. It depends on the learning signal and does not have the same function as the eligibility trace e_{jt}^{π} .

Data availability

The data that support the findings of this study are available from the authors upon reasonable request. Data for the TIMIT and ATARI benchmark tasks were published in previous works²⁹ with DOI [<https://doi.org/10.1613/jair.3912>] and ref.²⁰ with DOI [<https://doi.org/10.6028/nist.ir.4930>]. Data for the temporal credit assignment task are generated by a custom code provided in the abovementioned code repository.

Code availability

An implementation of e-prop solving the tasks of Figs. 2–5 is made public together with the publication of this paper https://github.com/IGITUGraz/eligibility_propagation.

Received: 9 December 2019; Accepted: 16 June 2020;

Published online: 17 July 2020

References

- LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).
- Allen Institute: Cell Types Database. © 2018 Allen Institute for Brain Science. Allen Cell Types Database, cell feature search. Available from: celltypes.brain-map.org/data (2018).
- Bellec, G., Salaj, D., Subramoney, A., Legenstein, R. & Maass, W. Long short-term memory and learning-to-learn in networks of spiking neurons. *32nd Conference on Neural Information Processing Systems* (2018).
- Huh, D. & Sejnowski, T. J. Gradient descent for spiking neural networks. *32nd Conference on Neural Information Processing Systems* (2018).
- Lillicrap, T. P. & Santoro, A. Backpropagation through time and the brain. *Curr. Opin. Neurobiol.* **55**, 82–89 (2019).
- Furber, S. B., Galluppi, F., Temple, S. & Plana, L. A. The SpiNNaker project. *Proc. IEEE* **102**, 652–665 (2014).
- Davies, M. et al. Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro* **PP**, 1–1 (2018).
- Williams, R. J. & Zipser, D. A learning algorithm for continually running fully recurrent neural networks. *Neural Comput.* **1**, 270–280 (1989).
- Sanhueza, M. & Lisman, J. The CAMKII/NMDAR complex as a molecular memory. *Mol. Brain* **6**, 10 (2013).
- Cassenaer, S. & Laurent, G. Conditional modulation of spike-timing-dependent plasticity for olfactory learning. *Nature* **482**, 47–52 (2012).
- Yagishita, S. et al. A critical time window for dopamine actions on the structural plasticity of dendritic spines. *Science* **345**, 1616–1620 (2014).
- Gerstner, W., Lehmann, M., Liakoni, V., Corneil, D. & Brea, J. Eligibility traces and plasticity on behavioral time scales: experimental support of neohbbian three-factor learning rules. *Front. Neural Circuits* **12**, 53 (2018).
- Sajad, A., Godlove, D. C. & Schall, J. D. Cortical microcircuitry of performance monitoring. *Nat. Neuro.* **22**, 265–274 (2019).
- Engelhard, B. et al. Specialized coding of sensory, motor and cognitive variables in VTA dopamine neurons. *Nature* **570**, 509–513 (2019).
- Roeper, J. Dissecting the diversity of midbrain dopamine neurons. *Trends Neurosci.* **36**, 336–42 (2013).
- Lillicrap, T. P., Cownden, D., Tweed, D. B. & Akerman, C. J. Random synaptic feedback weights support error backpropagation for deep learning. *Nat. Comm.* **7**, 13276 (2016).
- Nokland, A. Direct feedback alignment provides learning in deep neural networks. *30th Conference on Neural Information Processing Systems* (NIPS 2016).
- Samadi, A., Lillicrap, T. P. & Tweed, D. B. Deep learning with dynamic spiking neurons and fixed feedback weights. *Neural Comput.* **29**, 578–602 (2017).
- Clopath, C., Büsing, L., Vasilaki, E. & Gerstner, W. Connectivity reflects coding: a model of voltage-based STDP with homeostasis. *Nat. Neurosci.* **13**, 344–352 (2010).
- Garofolo, J. S., Lamel, L. F., Fisher, W. M., Fiscus, J. G. & Pallett, D. S. DARPA TIMIT acoustic-phonetic continuous speech corpus CD-ROM. NASA STI/Recon Technical Report, available at: <https://doi.org/10.6028/nist.ir.4930> (1993).
- Greff, K., Srivastava, R. K., Koutnik, J., Steunebrink, B. R. & Schmidhuber, J. LSTM: a search space odyssey. *IEEE TNNLS* arXiv:1503.04069v2 [cs.NE] (2017).
- Graves, A., Mohamed, A.-R. & Hinton, G. Speech recognition with deep recurrent neural networks. *ICASSP* pp. 6645–6649 (2013).
- Morcos, A. S. & Harvey, C. D. History-dependent variability in population dynamics during evidence accumulation in cortex. *Nat. Neurosci.* **19**, 1672–1681 (2016).
- Kappel, D., Legenstein, R., Habenschuss, S., Hsieh, M. & Maass, W. A dynamic connectome supports the emergence of stable computational function of neural circuits through reward-based learning. *eNeuro* **5**, ENEURO.0301-17(2018).
- Bartunov, S. et al. Assessing the scalability of biologically-motivated deep learning algorithms and architectures. In *Advances in Neural Information Processing Systems*. arXiv:1807.04587 [cs.LG] (2018).
- Mnih, V. et al. Asynchronous methods for deep reinforcement learning. In *ICML*, 1928–1937 (2016).
- Sutton, R. S. & Barto, A. G. Reinforcement Learning: An Introduction (MIT press, 2018).
- Frémaux, N. & Gerstner, W. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Front. Neural Circuits* **9**, 85 (2016).
- Bellemare, M. G., Naddaf, Y., Veness, J. & Bowling, M. The arcade learning environment: an evaluation platform for general agents. *JAIR* **47**, 253–279 (2013).
- Runyan, C. A., Piasini, E., Panzeri, S. & Harvey, C. D. Distinct timescales of population coding across cortex. *Nature* **548**, 92–96 (2017).
- Bellec, G. et al. Biologically inspired alternatives to backpropagation through time for learning in recurrent neural nets. Preprint at <http://arxiv.org/abs/1901.09049> (2019).
- Gilra, A. & Gerstner, W. Predicting non-linear dynamics by stable local learning in a recurrent spiking neural network. *Elife* **6**, e28295 (2017).
- Thalmeier, D., Uhlmann, M., Kappen, H. J. & Memmesheimer, R.-M. Learning universal computations with spikes. *PLoS Comput. Biol.* **12**, e1004895 (2016).
- Alemi, A., Machens, C. K., Deneve, S. & Slotine, J.-J. Learning nonlinear dynamics in efficient, balanced spiking networks using local plasticity rules. In *32 AAAI Conference on Artificial Intelligence* (2018).
- Nicola, W. & Clopath, C. Supervised learning in spiking neural networks with force training. *Nat. Commun.* **8**, 2208 (2017).
- Sussillo, D. & Abbott, L. F. Generating coherent patterns of activity from chaotic neural networks. *Neuron* **63**, 544–557 (2009).
- Abbott, L. F., DePasquale, B. & Memmesheimer, R.-M. Building functional networks of spiking model neurons. *Nat. Neurosci.* **19**, 350 (2016).
- Ingrosso, A. & Abbott, L. Training dynamically balanced excitatory-inhibitory networks. *PLoS ONE* **14**, e0220547 (2019).
- Kim, C. M. & Chow, C. C. Learning recurrent dynamics in spiking networks. *eLife* **7**, e37124 (2018).
- Zenke, F. & Ganguli, S. Superspike: Supervised learning in multilayer spiking neural networks. *Neural Comput.* **30** (2018).
- Shrestha, S. B. & Orchard, G. Slayer: Spike layer error reassignment in time. *32nd Conference on Neural Information Processing Systems* (NeurIPS 2018).
- Neftci, E. O., Augustine, C., Paul, S. & Detorakis, G. Event-driven random back-propagation: enabling neuromorphic deep learning machines. *Front. Neurosci.* **11**, 324 (2017).
- Kaiser, J., Mostafa, H. & Neftci, E. Synaptic plasticity dynamics for deep continuous local learning. *Front. Neurosci.* **14**, 424 (2020).
- Emre O. Neftci, F. Z., Hesham Mostafa. Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine* **36**, 51–63 (2019).
- Bengio, Y., Léonard, N. & Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. Preprint at <https://arxiv.org/abs/1308.3432> (2013).
- Courbariaux, M., Hubara, I., Soudry, D., El-Yaniv, R. & Bengio, Y. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. Preprint at <https://arxiv.org/abs/1602.02830> (2016).
- Marshall, O., Cho, K. & Savin, C. A unified framework of online learning algorithms for training recurrent neural networks. Preprint at <https://arxiv.org/abs/1907.02649> (2019).
- Mujika, A., Meier, F. & Steger, A. Approximating real-time recurrent learning with random kronecker factors. *Machine Learning* arXiv:1805.10842 [cs.LG] (2018).
- Tallec, C. & Ollivier, Y. Unbiased online recurrent optimization. *ICLR* (2018).
- Roth, C., Kanitscheider, I. & Fiete, I. Kernel rnn learning (kernel). *ICLR* (2019).
- Murray, J. M. Local online learning in recurrent networks with random feedback. *eLife* **8**, pii: e43299 (2019).

52. Jaderberg, M. et al. Decoupled neural interfaces using synthetic gradients. Preprint at <https://arxiv.org/abs/1608.05343> (2016).
53. Gerstner, W., Kistler, W. M., Naud, R. & Paninski, L. Neuronal dynamics: from single neurons to networks and models of cognition (Cambridge University Press, 2014).
54. Pozzorini, C., Naud, R., Mensi, S. & Gerstner, W. Temporal whitening by power-law adaptation in neocortical neurons. *Nat. Neurosci.* **16**, 942–8 (2013).
55. Esser, S. K. et al. Convolutional networks for fast, energy-efficient neuromorphic computing. *PNAS* **113**, 11441–11446 (2016).
56. Werbos, P. J. Backpropagation through time: what it does and how to do it. *Proceed. IEEE* **78**, 1550–1560 (1990).
57. Roelfsema, P. R. & Holtmaat, A. Control of synaptic plasticity in deep cortical networks. *Nat. Rev. Neurosci.* **19**, 166–180 (2018).
58. Pozzi, I., Bohtë, S. & Roelfsema, P. A biologically plausible learning rule for deep learning in the brain. Preprint at <https://arxiv.org/abs/1811.01768> (2018).

Acknowledgements

This research/project was supported by the Human Brain Project (grant agreement number 785907), the SYNCH project (grant agreement number 824162) of the European Union and a grant from Intel. We gratefully acknowledge the support of NVIDIA Corporation with the donation of the Quadro P6000 GPU used for this research. Computations were carried out on the Human Brain Project PCP Pilot Systems at the Juelich Supercomputing Centre, which received co-funding from the European Union (grant agreement number 604102) and on the Vienna Scientific Cluster (VSC). We thank Thomas Bohnstingl, Wulfram Gerstner, Christopher Harvey, Martin Vinck, Jason MacLean, Adam Santoro, Christopher Summerfield, and Yuqing Zhu for helpful comments on an earlier version of the manuscript. Special thanks go to Arjun Rao for letting us use his code for the regularization of membrane voltages.

Author contributions

G.B., F.S., A.S., and W.M. conceived the work, G.B., F.S., A.S., E.H., and D.S. carried out experiments, G.B., F.S., A.S., E.H., D.S., R.L., and W.M. contributed to the writing of the paper.

Competing interests

The authors declare no conflict of interest.

Additional information

Supplementary information is available for this paper at <https://doi.org/10.1038/s41467-020-17236-y>.

Correspondence and requests for materials should be addressed to W.M.

Peer review information *Nature Communications* thanks the anonymous reviewer(s) for their contribution to the peer review of this work. Peer reviewer reports are available.

Reprints and permission information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2020

Supplementary materials for: A solution to the learning dilemma for recurrent networks of spiking neurons

Guillaume Bellec^{1,◦}, Franz Scherr^{1,◦}, Anand Subramoney¹,

Elias Hajek¹, Darjan Salaj¹, Robert Legenstein¹, Wolfgang Maass^{1,*}

¹Institute of Theoretical Computer Science, Graz University of Technology,
Inffeldgasse 16b, Graz, Austria

◦ Equal contributions.

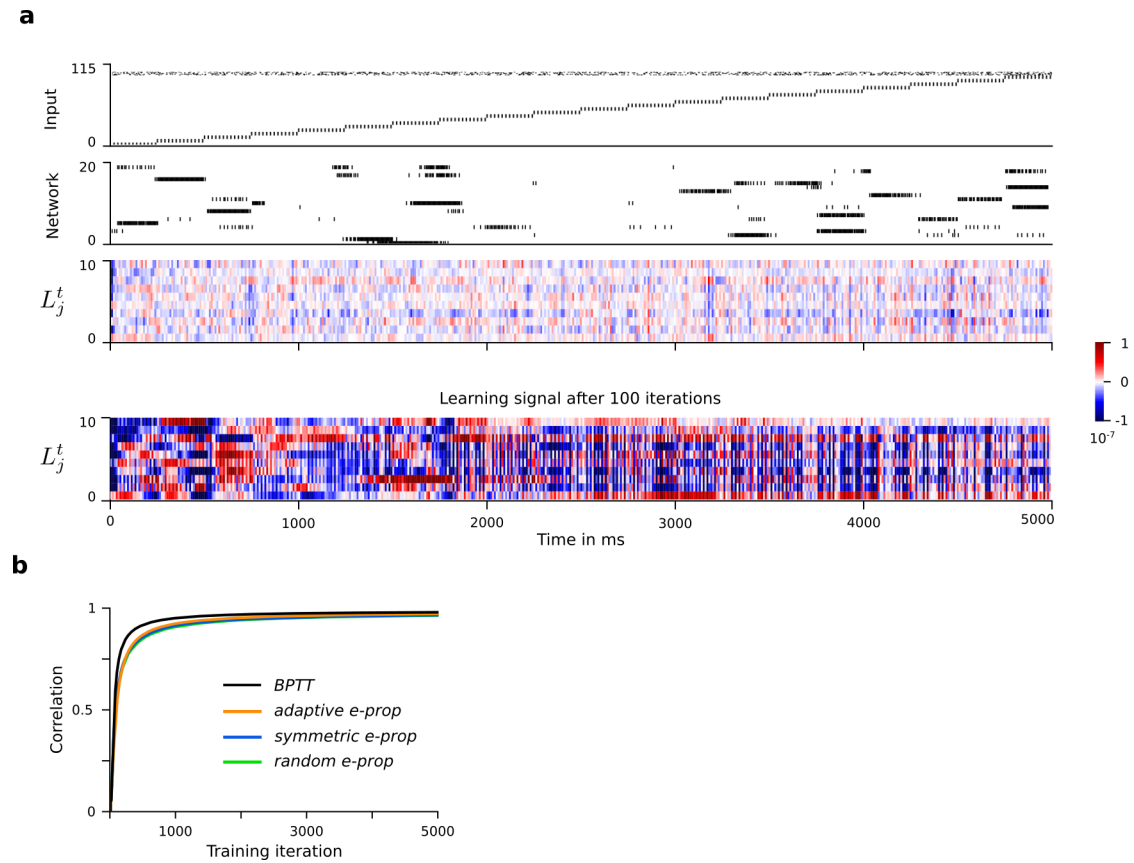
* To whom correspondence should be addressed; E-mail: maass@igi.tugraz.at.

Contents

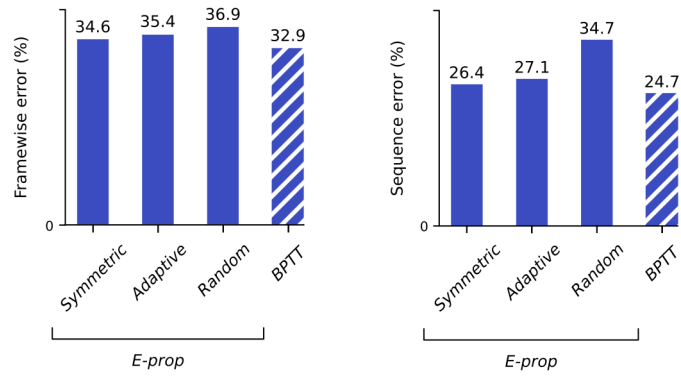
Supplementary Figures	3
Figure 1 Performance comparison of BPTT and e-prop on an extension of the episodic memory task from [1]	3
Figure 2 Comparison of learning algorithms for training LSNNs on the TIMIT task	4
Figure 3 Performance of e-prop on the framewise TIMIT task for LSNNs without recurrent connections	5
Figure 4 LSTM networks trained with BPTT and e-prop on the TIMIT task	6
Figure 5 Performance of reward-based random e-prop and BPTT for the RL version of the task from Figure 3	7
Figure 6 Impact of the length of the simulation time step on the learning performance of e-prop for the task of Figure 3	8
Figure 7 Evaluation of several variants of random e-prop	8
Figure 8 Test of a prediction of our analysis when e-prop is likely to approximate the performance of BPTT, and when not	10
Figure 9 Histogram of membrane potentials	11
Supplementary Tables	12
Table 1 Table of hyperparameters used in Atari tasks	12
Supplementary Notes	13
1 Eligibility traces	13
Eligibility traces for LSNNs with membrane potential reset	13
Eligibility traces for LSNNs with non-uniform synaptic delays	14
2 Optimization and regularization procedures	14
Optimization procedure	14
Firing rate regularization for LSNNs	15
Adaptive e-prop and weight decay regularization	15
Voltage regularization	15
Optimization with rewiring for sparse network connectivity	16
3 Supervised learning with e-prop	17
Synaptic plasticity rules for e-prop in supervised learning	17
Simulation details: framewise phoneme classification (Figure 2)	18
Simulation details: phoneme sequence recognition with CTC (Figure 2)	19

Applying e-prop to an episodic memory task	20
Simulation details: task where temporal credit assignment is difficult (Figure 3)	21
4 Applying supervised learning with e-prop to artificial neural networks (LSTM networks)	22
Phoneme recognition with LSTM networks and e-prop	22
Simulation details: phoneme recognition with LSTM networks (Supplementary Figure 4)	23
Simulation details: phoneme sequence recognition with CTC and LSTM (Supplementary Figure 4)	23
LSTM network model	23
Eligibility traces for LSTM units	24
5 Reward-based e-prop: Application of e-prop to deep RL	25
Synaptic plasticity rules for reward-based e-prop	25
Simulation details: RL version of the task where credit assignment is difficult (Supplementary Figure 5)	28
Simulation details: Atari task (Figures 4 and 5)	28
6 Evaluation of four variations of <i>e-prop</i> (Supplementary Figure 7)	30
truncated eligibility trace for LIF neurons	30
Global broadcast weights	30
Temporally local broadcast weights	30
Replacing the eligibility trace by the corresponding term of the Clopath rule	31
Simulation details: pattern generation task	31

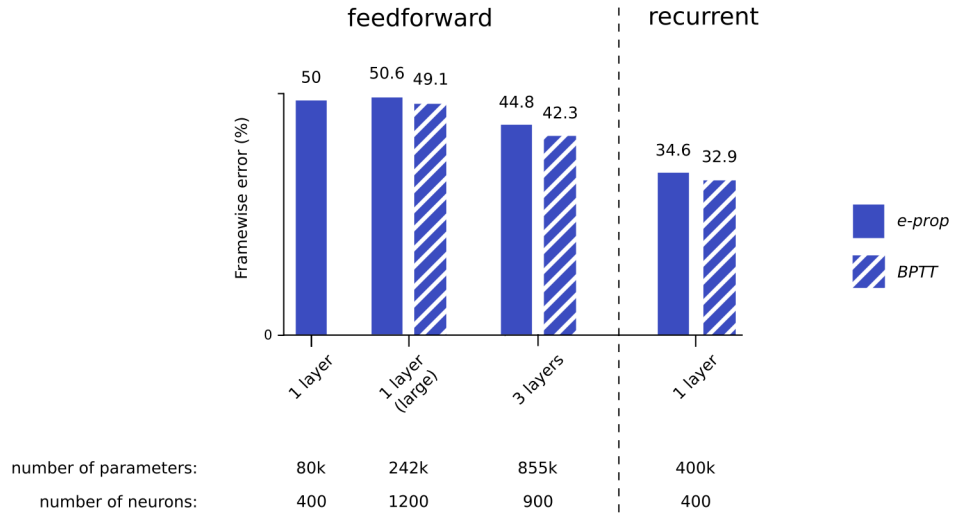
Supplementary Figures



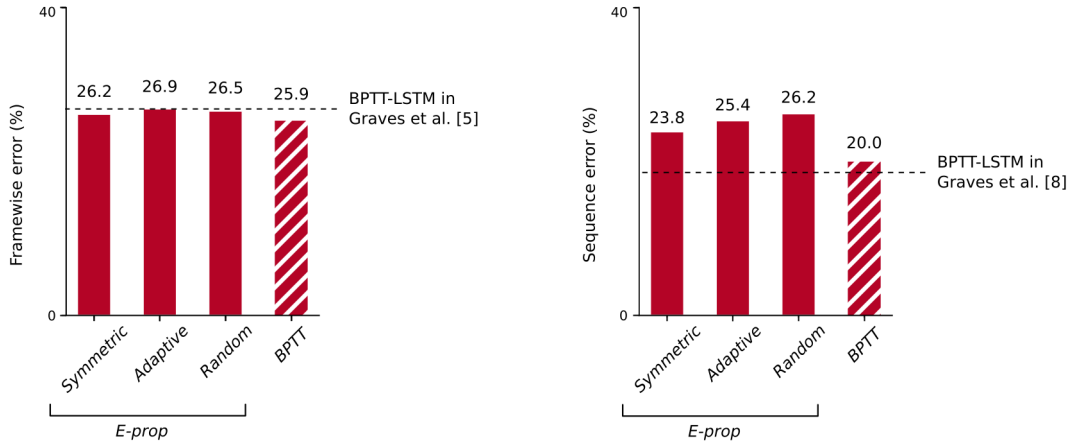
Supplementary Figure 1: **Performance comparison of BPTT and e-prop on an extension of the episodic memory task from [1]**. The LSNN had to generate at each time step the values of all pixels that described the video frame of the movie at that time step, see section 3. **a)** Input spikes, network activity (for 20 sample neurons), learning signals, and network outputs (at 1s and 4s, shown at the top) of an LSNN after 1000 training iterations. For comparison we also show learning signals after just 100 iterations, where their amplitude is still large. **b)** Performance of BPTT and e-prop.



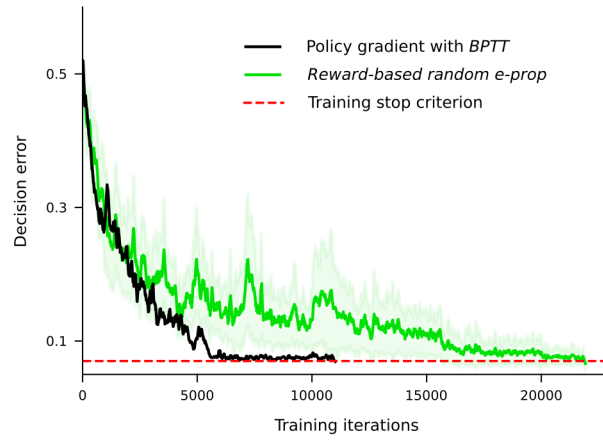
Supplementary Figure 2: **Comparison of learning algorithms for training LSNNs on the TIMIT task.** Performance of BPTT and the three versions of e-prop on frame-wise phoneme classification (left) and for phoneme sequence recognition (right).



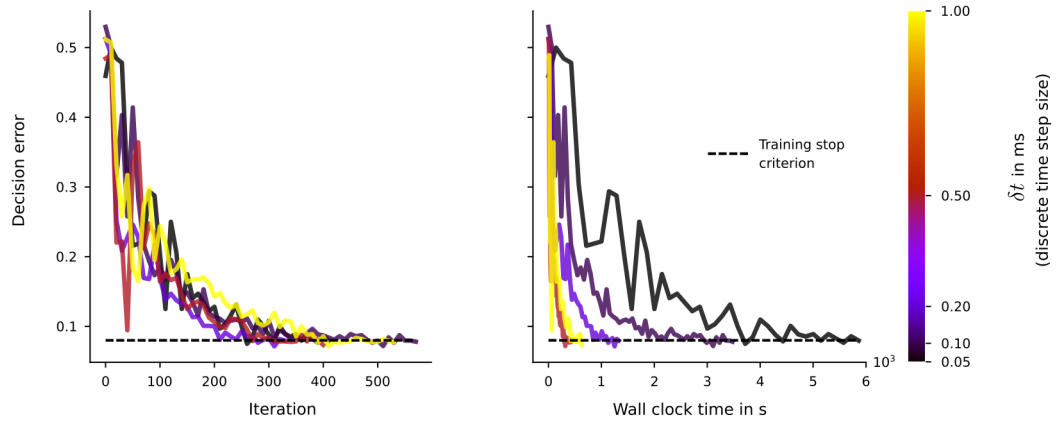
Supplementary Figure 3: **Performance of e-prop on the framewise TIMIT task for variations of the LSNN from Figure 2 without recurrent connections (left of dashed line).** The result of the left-most bar was achieved with the same hyperparameters as used in Figure 2. For the other feedforward architectures we modified hyperparameters to optimize the performance. The results of Figure 2 c) for LSNNs with recurrent connections are redrawn on the right of the dashed line. One sees that recurrent connections are essential for achieving good performance on this task.



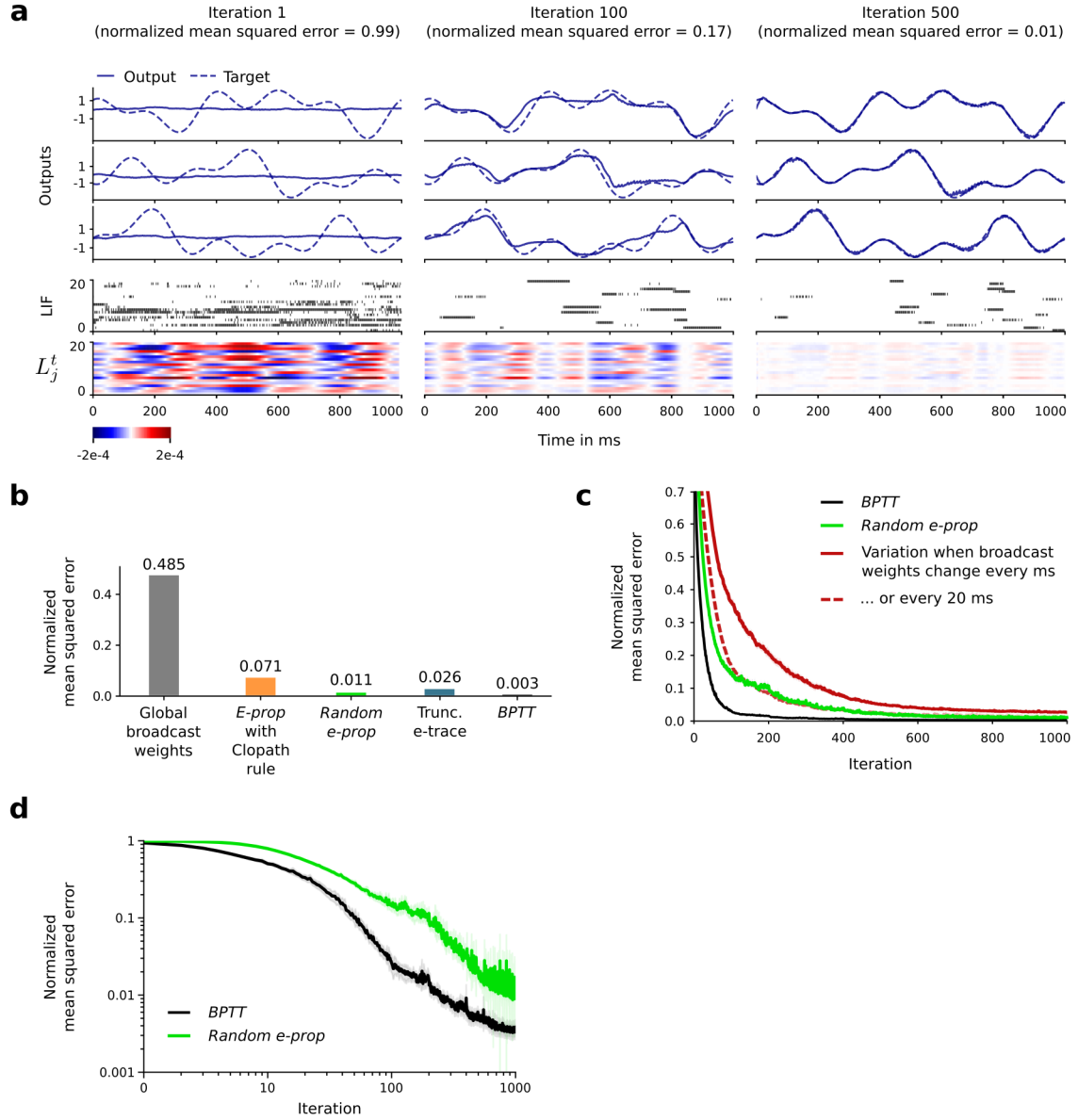
Supplementary Figure 4: **LSTM networks trained with BPTT and e-prop on the TIMIT task.** Performance of BPTT and the three versions of e-prop on frame-wise phoneme classification (left) and for phoneme sequence recognition (right). One sees that e-prop approximates BPTT performance also for non-spiking neural networks. The BPTT baselines aim at reproducing the results obtained with LSTM networks in [2] and [3]. The network architectures and audio pre-processing settings are taken from [2, 4] for framewise phoneme classification and from [3] for phoneme sequence transcription. In comparison to the BPTT-LSTM baselines that we could achieve in this way, 26.9% framewise error rate was reported in [2] and 18.6% sequence error rate was reported in [3].



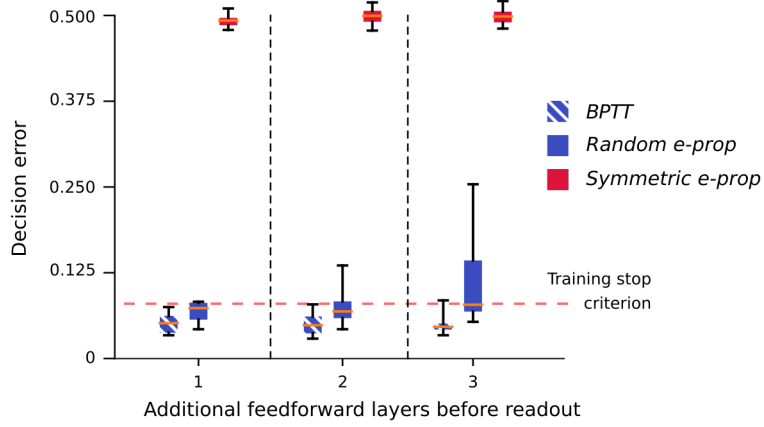
Supplementary Figure 5: **Performance of reward-based random e-prop and BPTT for the RL version of the task from Figure 3, applied to an LSNN consisting of 50 LIF and 50 ALIF neurons.**



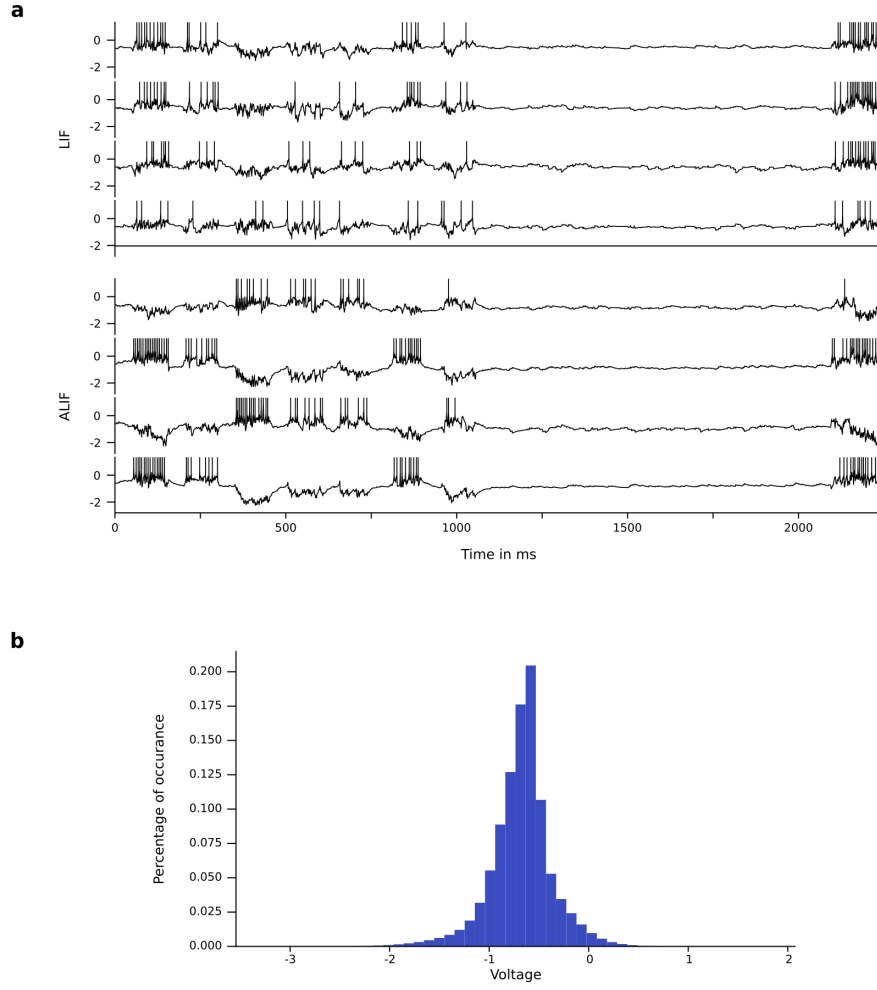
Supplementary Figure 6: **Impact of the length of the simulation time step on the learning performance of e-prop for the task of Figure 3 (simplified version with 5 cues instead of 7).** Decision error for this task is shown as function of the number of training iterations, and as function of wall clock time. The results are averaged over 5 different seeds. One sees that the length of the simulation time step has no visible impact on the learning performance of e-prop. However, smaller simulation time steps lead to substantially larger simulation time



Supplementary Figure 7: **Evaluation of several variants of random e-prop** **a)** The task is a classical benchmark task for learning in recurrent SNNs: learning to generate a target pattern, extended here to the challenge to simultaneously learn to generate 3 different patterns, which makes credit assignment for errors more difficult. Learning performance with random e-prop is shown after training for 1, 100, 500 s. **b)** Normalized mean squared error of several learning algorithms for this task after 500 s of training. “Clopath rule” denotes a replacement of the eligibility trace of random e-prop by a corresponding term proposed in [5] based on experimental data. **c)** Learning curves for variations of random e-prop with temporally local broadcast weights. Results were averaged over 20 runs. **d)** Comparison of learning curves for BPTT and random e-prop with logarithmic scale.



Supplementary Figure 8: **Test of a prediction of our analysis when e-prop is likely to approximate the performance of BPTT, and when not.** We blocked in the learning task of Figure 3 route i for gradient information by inserting 1 to 3 feedforward layers (completely connected from one layer to the next) between the RSNN and the readouts y of Figure 2a. All neurons in the RSNN were connected to the first feedforward layer, and all neurons in the last layer to the readout neurons. The decision error is reported for a distribution of runs with different initialization weights. The upper and lower whiskers are the minimum and the maximum, the orange bar is the median and the box extends from the lower to the upper quartile of the data. As predicted, the feedforward layers drastically reduce the performance of symmetric e-prop, but not of BPTT. Thus for successful training of a RSNN by symmetric e-prop it is essential that all or most neurons in the RSNN have a synaptic connection to readout neurons. Random e-prop works still quite well, but the high variance of its performance for 3 intermediate feedforward layers suggests some learning instability. See section 3 for details of this experiment.



Supplementary Figure 9: **Further details on membrane potentials of neurons in the network of Figure 3 a.** Traces of voltage – firing threshold ($v_j^t - A_j^t$) for 4 randomly selected LIF and 4 ALIF neurons. **b)** Histogram of these values for all neurons in the network and all time points t .

Supplementary Tables

	Pong	Fishing Derby
τ_m	15 ms	10 ms
v_{th}	1	1
τ_a	70 ms	70 ms
β	0.1	0.1
τ_{out}	2 ms	4 ms
$v_{th,CNN}$	0.1	0.1
$\tau_{m,CNN}$	1 ms	1 ms
c_V	1	0.5
c_H	0.0025	0.006
c_{reg}	50	10
$c_{reg,V}$	10^{-4}	10^{-3}
ΔT	$\left(\left(\sqrt[3]{\frac{T_{max}}{1\ s} \cdot 1000} + 1 \right)^3 - \frac{T_{max}}{1\ s} \cdot 1000 \right) \text{ ms}$	1000 ms
First convolutional Layer		
Kernel structure	$8 \times 8 \times 1 \times 8$	$8 \times 8 \times 1 \times 16$
Stride	4	4
Padding	VALID	SAME
Number of neurons	3200	7056
Second convolutional Layer		
Kernel structure	$4 \times 4 \times 8 \times 8$	$4 \times 4 \times 16 \times 32$
Stride	1	2
Padding	VALID	SAME
Number of neurons	2312	3872

Supplementary Table 1: Table of hyperparameters used in Atari tasks

Supplementary Notes

1 Eligibility traces

Eligibility traces have been introduced in section “Mathematical basis for e-prop” in Results. Here, we provide further information on eligibility traces. First, we discuss an alternative view on eligibility traces as derivatives. Second, we extend our treatment of eligibility traces for LSNNs in Methods to include non-uniform synaptic delays.

Viewing eligibility traces as derivatives

The notion of derivative $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ from Equation (14) in Methods quantifies the influence of an infinitesimal change of W_{ji} on the hidden state $\mathbf{h}_j^t$ through the internal processes of neuron j and the spikes of neuron i . Unlike the partial derivative $\frac{\partial \mathbf{h}_j^t}{\partial W_{ji}}$ it does not only take into account the update of the hidden state at time step t , but considers instead the full neuron history. In comparison to the total derivative $\frac{d\mathbf{h}_j^t}{dW_{ji}}$ it is only aware of the activity of neurons i and j . Below we provide a formal explanation why the eligibility traces and the eligibility vectors can be viewed as derivatives.

One also arrives at the Equation (13) in Methods if one computes the total derivative $\frac{d\mathbf{h}_j^t}{dW_{ji}}$ in a variation of the computational graph of Figure 6 where the function $\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, \tilde{\mathbf{z}}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$ receives a vector $\tilde{\mathbf{z}}^{t-1}$ that is equal to $\mathbf{z}^{t-1}$ but considered as a constant (i.e. the gradients are stopped at this node in the computational graph). Hence one can define the notation $\left[\frac{dA}{dB}\right]_{\text{local}}$ as a total derivative $\frac{dA}{dB}$ in this altered computational graph. To prove this equivalence one follows a derivation similar to the one detailed after equation (15) in Methods but in the altered computational graph: the total derivative $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ expands as $\sum_{t \leq t'} \left[\frac{d\mathbf{h}_j^{t'}}{d\mathbf{h}_j^t}\right]_{\text{local}} \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}$. Viewing that $\left[\frac{d\mathbf{h}_j^t}{d\mathbf{h}_j^{t-1}}\right]_{\text{local}} = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}}$ in this computational graph, one recognizes that $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ is the eligibility vector given in Methods, Equation (14). Equation (13) follows since $e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \epsilon_{ji}^t = \left[\frac{dz_j^t}{dW_{ji}}\right]_{\text{local}}$. In fact this definition allows the extension of this new local derivative to other quantities and one can summarize symmetric e-prop as the replacement of $\frac{dE}{dW_{ji}}$ by $\left[\frac{dE}{dW_{ji}}\right]_{\text{local}}$ in stochastic gradient descent.

Eligibility traces for LSNNs with membrane potential reset

The eligibility traces derived in the methods do not take the reset term into account. We derive here the eligibility traces that can correct for this. Note however that we did not observe an improvement when using this more complex model on the phoneme recognition and the task where temporal credit assignment is difficult.

Eligibility traces for LIF neurons: When taking into account the reset, the partial derivative $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ becomes $\alpha - v_{\text{thr}} \psi_j^t$ instead of α and, accordingly to Equation (14) from Methods, the eligibility vector can be computed with the recursive formula: $\epsilon_{ji}^{t+1} = (\alpha - \beta \psi_j^t) \epsilon_{ji}^t + z_j^t$.

Eligibility traces for ALIF neurons: According to the dynamics of the ALIF neurons defined in Equations (6)–(10) in Methods one coefficient differs in the matrix $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} \in \mathbb{R}^{2 \times 2}$ as soon as one takes the reset into account. The coefficient $\frac{\partial v_j^t}{\partial a_j^t}$ was 0 without reset and becomes now $v_{\text{thr}} \beta \psi_j^t$.

Overall the full derivative $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ is then equal to:

$$\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} = \begin{pmatrix} \alpha - v_{\text{thr}} \psi_j^t & v_{\text{thr}} \beta \psi_j^t \\ \psi_j^t & \rho - \beta \psi_j^t \end{pmatrix}. \quad (1)$$

Even-though e-prop can be implemented as such, the recursive propagation of the eligibility vector in Equation (14) from Methods cannot be written in the form of two separable Equations as done in Equations (24) and (25) in Methods. Hence, we preferred to ignore the reset in Methods to provide more interpretable Equations for eligibility traces.

Eligibility traces for LSNNs with non-uniform synaptic delays

In our derivation of eligibility traces for LSNNs, we used uniform synaptic delays to ease notation. Here, we detail how e-prop can be extended to non-uniform delays of several ms. Let the delay of a synapse from neuron i to j be denoted by $c(j, i) > 0$. Similarly, let $d(j, i) \geq 0$ be the delay of a synapse that connects an input neuron i with neuron j . Using this definition, the dynamics of the membrane potential, see Equation (6) in Methods, is written as:

$$v_j^{t+1} = \alpha v_j^t + \sum_{i \neq j} W_{ji}^{\text{rec}} z_i^{t-c(j,i)} + \sum_i W_{ji}^{\text{in}} x_i^{t-d(j,i)} - z_j^t v_{\text{th}}. \quad (2)$$

Like in the uniform delay case, we obtain $\frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$. The difference for arbitrary delays becomes visible in $\frac{\partial v_j^t}{\partial W_{ji}^{\text{rec}}} = z_i^{t-c(j,i)}$ and in $\frac{\partial v_j^t}{\partial W_{ji}^{\text{in}}} = x_i^{t-d(j,i)}$. For recurrent weights, the component of the eligibility vector associated to the membrane potential is hence:

$$\epsilon_{ji,v}^t = \sum_{t' \leq t-c(j,i)} z_i^{t'} = \bar{z}_i^{t-c(j,i)}. \quad (3)$$

As the dynamics of the threshold adaptation is unchanged, the update of $\epsilon_{ji,a}^t$ remains as given in Equation (24) in Methods. We obtain an eligibility trace

$$e_{ji}^t = \psi_j^t \left(\bar{z}_i^{t-c(j,i)} - \beta \epsilon_{ji,a}^t \right). \quad (4)$$

Analogously, we obtain the corresponding eligibility trace for input synapses by replacing z_i^t and $c(j, i)$ with x_i^t and $d(j, i)$ respectively.

2 Optimization and regularization procedures

Here, we discuss how optimization of networks was implemented and techniques that were used to regularize networks.

Optimization procedure

For e-prop and for BPTT, the weights were updated once after a batch of training trials. For simplicity, all the weight updates $\Delta W_{ji}^{\text{rec}}$ are written for the most basic version of stochastic gradient descent ($\Delta W_{ji}^{\text{rec}} = -\eta \widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}$, where $\widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}$ is the gradient estimate) in this article. In practice, we used Adam [6] to boost stochastic gradient descent. We refer to [6] for the computation of the weight updates that result from the gradient estimates.

Firing rate regularization for LSNNs

To ensure a low firing rate in LSNNs, we added a regularization term E_{reg} to the loss function E . This regularization term had the form:

$$E_{\text{reg}} = \frac{1}{2} \sum_j (f_j^{\text{av},t} - f^{\text{target}})^2, \quad (5)$$

where f^{target} is a target firing rate and $f_j^{\text{av}} = \frac{1}{Tn_{\text{trial}}} \sum_t z_j^t$ is the average firing rate of neuron j . Here, the sum runs over the time steps of all the n_{trial} trials between two weight updates. To derive the plasticity rule that implements this regularization, we follow Equation (27) in Methods. The partial derivative of the regularization loss has the form:

$$\frac{\partial E_{\text{reg}}}{\partial z_j^t} = \frac{1}{Tn_{\text{trial}}} (f_j^{\text{av}} - f^{\text{target}}). \quad (6)$$

Inserting this expression into Equation (27) in Methods, we obtain the plasticity rule that implements the regularization:

$$\Delta W_{ji}^{\text{rec}} = \eta c_{\text{reg}} \sum_t \frac{1}{Tn_{\text{trial}}} (f^{\text{target}} - f_j^{\text{av}}) e_{ji}^t, \quad (7)$$

where c_{reg} is a positive coefficient that controls the strength of the regularization. This plasticity rule is applied simultaneously together with the plasticity rule that minimizes the loss E . Note that this weight update fits the e-prop framework provided by Equation (1) from Results with a learning signal L_j^{reg} proportional to $f^{\text{target}} - f_j^{\text{av}}$ available locally at neuron j . The learning signal L_j^{reg} can be computed locally and added to the task-specific learning signal L_j^t . A variant of this regularization replaces the average firing rate f_j^{av} with a running average $f_j^{\text{av},t}$ that accumulates the spikes since the last weight update. It can achieve the same effect with less resources when implemented online.

Adaptive e-prop and weight decay regularization

In adaptive e-prop all neurons are receiving learning signals with random broadcast weights B_{jk} as in random e-prop. Yet, if neuron j sends activity to the output neuron k via output weights W_{kj}^{out} , the broadcast weights B_{jk} are subsequently adapted. This was implemented by applying an identical weight update to the broadcast and the output weights, i.e. $\Delta B_{jk} = \Delta W_{kj}^{\text{out}}$ and by subtracting $c_{\text{decay}} \cdot W_{kj}^{\text{out}}$ (resp. $c_{\text{decay}} \cdot B_{jk}$) from the weight W_{kj}^{out} (resp. B_{jk}) after each weight update, where $c_{\text{decay}} > 0$ is the regularization factor (see specific experiments for the value of c_{decay}). This weight decay in combination with the mirroring of the weight updates has the effect that, despite different initialization, the output weights and the adaptive broadcast weights converge to similar values. The remaining difference of performance between symmetric and adaptive e-prop reported in Supplementary Figure 2 and Supplementary Figure 4 may be explained by the different initializations.

Adaptive e-prop can be viewed as that version of e-prop that exploits all types of biologically plausible learning signals that are available at individual neurons, apart from reward prediction errors or signals from a separately trained RSSN that sends learning signals, see Figure 3 of [7].

Voltage regularization

One advantage of a gradient-based learning method is that additional terms can be added to the loss function in order to enforce a particular network behavior. We utilized this possibility to regularize the membrane voltage of network neurons, using an idea of Arjun Rao. This regularization reduced excessive membrane voltages (which can otherwise even exceed the threshold voltage during the neuron's refractory period) and led to sparser and more irregular firing activity, similar

as in biological data. In particular, we used a regularization term that penalized membrane potentials higher than the threshold voltage A_t for ALIF neurons (resp. v_{thr} for LIF) or lower than $-A_t$ for ALIF neurons (resp. $-v_{\text{thr}}$ for LIF). We add a regularization loss of the form:

$$E_{V,\text{reg}} = E_{\text{reg},V+} + E_{\text{reg},V-} = \frac{1}{2} \sum_t \left([v_j^t - A_j^t]^+ \right)^2 + \left([-v_j^t - A_j^t]^+ \right)^2 \quad (8)$$

for ALIF neurons (replace A_t by v_{thr} for LIF neurons), where $[\cdot]^+$ is the rectified linear function. Unlike other loss functions, $E_{V,\text{reg}}$ depends directly on the voltage and not on the spikes. Hence instead of the full e-prop formula we use that the cost function is already local to neuron j , so that we can derive the weight update as follows (we consider only the first part $E_{\text{reg},V+}$ of the loss function to simplify the analysis):

$$\Delta W_{ji} = -\eta c_{\text{reg},V} \sum_t \left[\frac{dE_{\text{reg},V+}}{dW_{ji}} \right]_{\text{local}} \quad (9)$$

$$= -\eta c_{\text{reg},V} \sum_t \frac{\partial E_{\text{reg},V+}}{\partial \mathbf{h}_j^t} \epsilon_j^t \quad (10)$$

$$= -\eta c_{\text{reg},V} \sum_t [v_j^t - A_j^t]^+ (\epsilon_{ji,v}^t - \epsilon_{ji,a}^t) \quad (11)$$

$$= -\eta c_{\text{reg},V} \sum_t [v_j^t - A_j^t]^+ (\bar{z}_i^{t-1} - \epsilon_{ji,a}^t) . \quad (12)$$

Optimization with rewiring for sparse network connectivity

Due to limited resources, neural networks in the brain and in neuromorphic hardware are sparsely connected. In addition, the connectivity structure of brain networks is dynamic, with synaptic connections being added and deleted on the time scale of hours or days, which was shown to help the network to use the limited connectivity resources in an optimal manner [8]. In order to test whether e-prop is compatible with synaptic rewiring, we combined it with DEEP R [9]. DEEP R is based on a model for synaptic rewiring in the brain [8] and allows to rewire sparse neural network models during training with gradients descent. The algorithm minimizes the loss function E subject to a constraint on the total number of connected synapses. To do so, each synaptic weight W_{ji} is assigned a fixed sign s_{ji} (it is defined to be excitatory or inhibitory) and an amplitude w_{ji} . Each potential synaptic connection can either be “active”, i.e., the synaptic connection is realized, or “dormant”, i.e., this potential connection is not realized.

For a dormant synaptic connection, the weight W_{ji} is set to be zero and the gradients and weight updates of the connection $i \rightarrow j$ are not computed. It means in e-prop that dormant synapses do not require eligibility traces. For an active connection, the weight is defined as $W_{ji} = s_{ji} w_{ji}$ and the weight amplitude is updated according to the update $\Delta w_{ji} = s_{ji} \Delta W_{ji} - \eta c_{L1}$ where ΔW_{ji} is the weight update given here by e-prop and $c_{L1} = 0.01$ is an $L1$ regularization coefficient. To update the network structure such that the set of active connections is optimized along side their synaptic weights, DEEP R proceeds as follows after each weight update:

- every active connection for which the amplitude becomes negative is set to be dormant,
- and some dormant connections are selected randomly and set to be active with $w_{ji} = 0$ such that the total number of active connection remains constant.

We define the synapse signs s_{ji} such that 80% of the neurons are excitatory and 20% are inhibitory. Despite the constraint on the neuron signs and the constraint that 90% of the synapses should remain dormant throughout the learning process, e-prop and rewiring solve the task where temporal credit assignment is difficult of Figure 3.

3 Supervised learning with e-prop

Synaptic plasticity rules for e-prop in supervised learning

Here, we derive synaptic plasticity rules that result from e-prop for supervised learning. We consider two cases: First, we derive plasticity rules for regression tasks, and second, for classification tasks.

We follow the scheme described by Equation (27) in Methods. Hence the loss gradients $\frac{dE}{dW_{ji}}$ are estimated using the approximation $\widehat{\frac{dE}{dW_{ji}}} \stackrel{\text{def}}{=} \sum_t \frac{\partial E}{\partial z_j^t} e_{ji}^t$. Given the eligibility traces that are derived in Methods for spiking neurons and Supplementary Note 4 for LSTM, what remains to be derived for each task is the expression of the relevant derivative $\frac{\partial E}{\partial z_j^t}$ and show that it can be computed online.

Regression tasks: Consider a regression problem with loss function $E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2$, targets $y_k^{*,t}$ and outputs y_k^t as defined in Equation (11) in Methods. The partial derivative $\frac{\partial E}{\partial z_j^t}$ takes the form:

$$E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2 \quad (13)$$

$$\frac{\partial E}{\partial z_j^t} = \sum_k W_{kj}^{\text{out}} \sum_{t' \geq t} (y_k^{t'} - y_k^{*,t'}) \kappa^{t'-t} . \quad (14)$$

This seemingly provides an obstacle for online learning, because the partial derivative is a weighted sum over future errors. But this problem can be resolved. Following Equation (1) in Results, the approximation $\widehat{\frac{dE}{dW_{ji}}}$ of the loss gradient is computed with e-prop as follows (we insert $\frac{\partial E}{\partial z_j^t}$ in place of the total derivative $\frac{dE}{dz_j^t}$):

$$\widehat{\frac{dE}{dW_{ji}}} = \sum_{t'} \frac{\partial E}{\partial z_j^{t'}} e_{ji}^{t'} \quad (15)$$

$$= \sum_{k,t'} W_{kj}^{\text{out}} \sum_{t \geq t'} (y_k^t - y_k^{*,t}) \kappa^{t-t'} e_{ji}^{t'} \quad (16)$$

$$= \sum_{k,t} W_{kj}^{\text{out}} (y_k^t - y_k^{*,t}) \underbrace{\sum_{t' \leq t} \kappa^{t-t'} e_{ji}^{t'}}_{\stackrel{\text{def}}{=} \tilde{e}_{ji}^t} , \quad (17)$$

where we changed the order of summations in the last line. The second sum indexed by t' is now over previous events that can be computed online. It is just a low-pass filtered version of the eligibility trace e_{ji}^t . With this additional filtering of the eligibility trace with a time constant equal to that of the leak of output neurons, we see that e-prop takes into account the latency between an event at time t' and its impact on later errors at time t within the integration time window of the output neuron. Hence, implementing weight updates with gradient descent and learning rate η , the plasticity rule resulting from e-prop is given by the Equation (28) in Methods. The gradient of the loss function with respect to the output weights $\frac{dE}{dW_{kj}^{\text{out}}}$ can be implemented online without relying on the theory of e-prop. The plasticity rule resulting from gradient descent is directly:

$$\Delta W_{kj}^{\text{out}} = -\eta \sum_t (y_k^t - y_k^{*,t}) \mathcal{F}_\kappa(z_j^t) . \quad (18)$$

Similarly the update of the bias of the output neurons is $\Delta b_k^{\text{out}} = -\eta \sum_t (y_k^t - y_k^{*,t})$.

Classification tasks: We assume that K target categories are provided in the form of a K -dimensional one-hot encoded vector $\pi^{*,t}$. To train recurrent networks in this setup, we replace the mean squared error by the cross entropy loss:

$$E = - \sum_{t,k} \pi_k^{*,t} \log \pi_k^t, \quad (19)$$

where the probability for class k predicted by the network is given as $\pi_k^t = \text{softmax}_k(y_1^t, \dots, y_K^t) = \exp(y_k^t) / \sum_{k'} \exp(y_{k'}^t)$. To derive the modified learning rule that results from this loss function E , we replace $\frac{\partial E}{\partial z_j^t}$ of Supplementary Equation (14) above with the corresponding one resulting from (19):

$$\frac{\partial E}{\partial z_j^t} = \sum_k W_{kj}^{\text{out}} \sum_{t' \geq t} (\pi_k^{t'} - \pi_k^{*,t'}) \kappa^{t'-t}. \quad (20)$$

Following otherwise the same derivation as above in Supplementary Equations (15)-(17), the plasticity rule in the case of classification tasks is given in Methods by Equation (29).

Similarly, one obtains the plasticity rule for the output connections, where the only difference between the cases of regression and of classification is that the output y_k^t and the target $y_k^{*,t}$ are replaced by π_k^t and $\pi_k^{*,t}$ respectively: $\Delta W_{kj}^{\text{out}} = -\eta \sum_t (\pi_k^t - \pi_k^{*,t}) \mathcal{F}_\kappa(z_j^t)$. The update of the bias of the output neurons is $\Delta b_k^{\text{out}} = -\eta \sum_t (\pi_k^t - \pi_k^{*,t})$.

Simulation details: framewise phoneme recognition task (Figure 2)

The goal of the frame-wise setup of the task is to classify audio-frames into phoneme classes. Every input sequence of audio-frames has a corresponding sequence of class labels of the same length, hence the model does not need to align the input sequence to the target sequence. This task has been widely adopted as a phoneme recognition benchmark for recurrent neural networks (RNNs).

Details of the network model: In Figure 2c the performance is reported for the same network architecture as used for LSTM networks in [2], it uses in particular a bi-directional architecture where the output of the LSNN is augmented by the output of a second LSNN that receives the input sequence in reverse time order. This reduces the error in comparison to the uni-directional case from 36.1% to 32.9% for LSNNs with BPTT. This improvement is qualitatively similar to what was previously reported for LSTM networks [2].

With the bi-directional architecture used in Figure 2c, each of the two networks consisted of 300 LIF neurons and 100 ALIF neurons. The neurons in the LSNN had a membrane time constant of $\tau_m = 20$ ms, an adaptation time constant of $\tau_a = 200$ ms, an adaptation strength of $\beta = 0.184$, a baseline threshold $v_{\text{th}} = 1.6$, and a refractory period of 2 ms.

We used 61 output neurons in total, one for each class of the TIMIT dataset. The membrane time constant of the output neurons was $\tau_{\text{out}} = 3$ ms. A softmax was applied to their output, resulting in the corresponding class probabilities. The network model had ≈ 0.4 million weights.

Details of the dataset preparation and of the input preprocessing: We followed the same task setup as in [4, 2]. The TIMIT dataset was split according to [10] into a training, validation, and test set with 3696, 400, and 192 sequences respectively. The input $\mathbf{x}^t$ was given as preprocessed audio that was obtained by the following procedure: Computation of 13 Mel Frequency Cepstral Coefficients (MFCCs) with a frame size of 10 ms on an input window of length 25 ms, computation of the first and the second derivatives of MFCCs, concatenation of all computed factors. The 39 input channels were mapped to the range $[0, 1]$ according to the minimum/maximum values in the training set.

In order to map the inputs into the temporal time domain of LSNNs, each preprocessed audio frame was fed as inputs $\mathbf{x}^t$ to the LSNN for 5 consecutive 1 ms steps.

Details of the learning procedure: All networks were trained for a maximum of 80 epochs, where we used early stopping to report the test error at the point of the lowest error on the validation set. Weight updates were implemented using Adam with default hyperparameters [6] except for ϵ_{Adam} , which was set to 10^{-5} . Gradients were computed using batches of size 32. We used L2 regularization in all networks by adding the term $10^{-5} \cdot \|W\|^2$ to the loss function, where W denotes all weights in the network. The learning rate was initialized to 0.01 and fixed during training. For random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with a mean of 0 and a variance of 1 and $1/n$ (n being the number of neurons in the LSNN) respectively. In adaptive e-prop, we used in addition to the weight decay described above L2 weight decay on output and broadcast weights according to 2 using a factor of $c_{\text{decay}} = 10^{-2}$. Firing rate regularization, as described in section 2, was applied with $c_{\text{reg}} = 50$.

Simulation details: phoneme sequence recognition with CTC (Figure 2)

We compared e-prop and BPTT on the task and the network architecture used in [3]. The essential building blocks of this architecture were also used in [11] for developing commercial software for speech-to-text transcriptions. In this architecture Connectionist Temporal Classification (CTC) is employed. This enabled us to train networks on unaligned sequence labeling tasks end-to-end. We considered the results of [3] that were obtained with three layers of bi-directional LSTM networks, CTC, and BPTT as a reference. We are aware that this configuration cannot be adapted to an online implementation easily, due to the usage of a bi-directional LSTM network and the CTC loss function. However, we believe that this task is still relevant to compare BPTT and e-prop because it is a well established benchmark for RNNs.

Details of the network model: The neurons were structured into 3 layers. The network was recurrently connected within a layer and had feedforward connections across layers. Each layer consisted of 80 LIF neurons and 720 ALIF neurons (9.1 million weights). The neurons in LSNNs had a membrane time constant of $\tau_m = 20$ ms, an adaptation time constant of $\tau_a = 500$ ms, an adaptation strength of $\beta = 0.074$, a baseline threshold $v_{\text{th}} = 0.2$, and a refractory period of 2 ms. Synaptic delays were randomly chosen from $\{1, 2\}$ ms with equal probability. The membrane time constant of output neurons was $\tau_{\text{out}} = 3$ ms.

E-prop with many layers of recurrent neurons: If one naively applies e-prop in such a configuration, the partial derivative $\frac{\partial E}{\partial z_j^t}$ is non-zero only if j belongs to the last layer, whereas earlier layers would not receive any learning signal. To avoid this, we connected all neurons in all layers of the RNN to the output neurons. Therefore, the outputs y_k^t of the RNN were given as $y_k^t = \sum_{t' \leq t} \kappa^{t-t'} \sum_l \sum_j W_{kj}^{\text{out},(l)} z_j^{(l),t'}$, where $z_j^{(l),t'}$ denotes the visible state of a neuron j within the layer l . As a result, the learning signals in the case of e-prop were non-zero for neurons in every layer.

E-prop with the CTC loss function: E_{CTC} is defined based on the log-likelihood of obtaining the sequence of labeled phonemes given the network outputs y_k^t . We refer to [12] for the formal definition of the probabilistic model. Equation (7.27) in [13] shows the gradient of the loss function E_{CTC} with respect to the activity of the outputs y_k^t that we denote as $\frac{dE}{dy_k^t}$. Using the linear relationship between the visible state $z_j^{(l),t}$ and the outputs y_k^t , we obtain that the partial derivative $\frac{\partial E_{\text{CTC}}}{\partial z_j^{(l),t}}$ that we need in order to find the learning signals used in e-prop, and are defined as $\sum_{t' \geq t} \kappa^{t'-t} \sum_k \frac{dE}{dy_k^{t'}} B_{jk}^{(l)}$. Here, $B_{jk}^{(l)}$ denote the broadcast weights to the layer l .

Details of the dataset preparation and of the input preprocessing: The TIMIT dataset was split in the same manner as in the frame-wise version of the task. The raw audio was preprocessed before it was provided as an input $\mathbf{x}^t$ to the network. This included the following

steps: computation of a Fourier-transform based filter-bank with 40 coefficients and an additional channel for the signal energy (with step size 10 ms and window size 25 ms), computation of the first and the second derivatives, concatenation of all computed factors, which totals to 123 input channels. Normalization over the training set was done in the same manner as in the frame-wise version of the task.

In order to map the inputs into the temporal time domain of LSNNs, each preprocessed audio frame was fed as inputs $\mathbf{x}^t$ to the LSNN for 5 consecutive 1 ms steps.

Details of the learning procedure: All models were trained for a total of 60 epochs, where gradients were computed using batches of 8 sequences. The learning rate was initialized to 10^{-3} and decayed every 15 epochs by a factor of 0.3. We used early stopping to report the test error, as in the previous task. Dropout was applied during training between the hidden layers and at the output neurons with a dropout probability of 0.3. As in the frame-wise setup, the weight updates were implemented using Adam with the default hyperparameters [6] except for $\epsilon_{\text{Adam}} = 10^{-5}$. For random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with a mean of 0 and a variance of 1 and $1/n$ (n denoting the number of neurons in the corresponding layer) respectively. In adaptive e-prop, we used L2 weight decay on output and broadcast weights according to 2 using a factor of $c_{\text{decay}} = 10^{-4}$. When the global norm of gradients $N_{\text{clip}} = \|\widehat{\frac{dE}{dW_{ji}^{\text{in}}}}\|^2 + \|\widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}\|^2 + \|\widehat{\frac{dE}{dW_{ji}^{\text{out}}}}\|^2$ was larger than 1, we scaled the gradients by a factor of $\frac{1}{N_{\text{clip}}}$. We used beam search decoding with a beam width of 100. As in [3], the networks were trained on all 61 phoneme labels but were then mapped to a reduced phoneme set (39 classes) for testing.

Applying e-prop to an episodic memory task

The FORCE training method [1] arguably defines the state of the art for training methods for RSNNs that do not need to backpropagate gradients through time. FORCE learning uses a synaptic plasticity rule that requires knowledge of the values of all synaptic weights in the network. This rule was not argued to be biologically plausible, but no other methods for training an RSNN to solve the task described below were known so far.

In order to compare e-prop to FORCE learning, we tested e-prop on the task to replay a movie segment that was used as a target $\mathbf{y}^{*,t}$ [1]. Specifically, it had to generate at each time step the values of all pixels that described the video frame of the movie at that time step. This episodic memory task was arguably the most difficult task for which an RSNN was previously trained in [1].

Here, we considered an extension to this task: An LSNN had to replay 1 out of 3 possible movies, where the desired movie index was provided as a cue to the network, see Supplementary Figure 1a. As in [1], the LSNN received also a clock-like input signal to indicate the current position in the movie. We show in Supplementary Figure 1b that an LSNN can be trained to solve this task with either one of the e-prop versions and that e-prop performs almost as well as BPTT.

Details of the network model: We used an LSNN that consisted of 700 LIF neurons and 300 ALIF neurons. Each neuron had a membrane time constant of $\tau_m = 20$ ms and a refractory period of 5 ms. ALIF neurons had a threshold adaptation time constant of 500 ms, and a threshold adaptation strength of $\beta = 0.07$. All neurons had a baseline threshold of $v_{\text{th}} = 0.62$. All 5544 output neurons had a membrane time constant of $\tau_{\text{out}} = 4$ ms.

Details of the dataset preparation and of the input scheme: We manually chose three movie clips from the Hollywood 2 dataset [14], which contained between 0 and 2 scene cuts*. The movie clips were clipped to a length of 5 seconds and spatially subsampled to a resolution of

*sceneclipautoautotrain00019.avi, sceneclipautoautotrain00061.avi, sceneclipautoautotrain00071.avi

66×28 pixels. Since our simulations used 1 ms as a discrete time step, we linearly interpolated between the frames of the original movie clips, which had a framerate of 25 frames per second. In total, we obtained a target signal with $66 \times 26 \times 3 = 5544$ dimensions, whose values were divided by a constant of 255, such that they fit in the range of $[0, 1]$.

The network received input from 115 input neurons, divided into 23 groups of 5 neurons. The first 20 groups indicated the current phase of the target sequence, similar to [1]. Neurons in group $i \in \{0, 19\}$ produced regular spike trains with a firing rate of 50 Hz during the time interval $[250 \cdot i, 250 \cdot i + 250)$ ms and were silent at other times. The remaining 3 groups encoded which movie had to be replayed, where each group was assigned to one of the three movies. To indicate a desired replay of one specific movie, each neuron in the corresponding group produced a Poisson spike train with a rate of 50 Hz and was silent otherwise.

Details of the learning procedure: For learning, we carried out 5 second simulations, where the network produced a 5544 dimensional output pattern. Gradients were accumulated for 8 successive trials, after which weight updates were applied using Adam with a learning rate of $2 \cdot 10^{-3}$ and default hyperparameters [6]. The movie to be replayed in each trial was selected with uniform probability. After every 100 weight updates (iterations), the learning rate was decayed by a factor of 0.95. For random e-prop, we used random broadcast weights B_{jk} that were sampled from a Gaussian distribution with a mean of 0 and a variance of 1. In adaptive e-prop we used L2 weight decay (see Supplementary Note 2 on weight decay) for the broadcast weights B_{jk} and the output weights W_{ji}^{out} with a factor of $c_{\text{decay}} = 0.001$. To avoid an excessively high firing rate, regularization, as described in Supplementary Note 2 on firing rate regularization, was applied with $c_{\text{reg}} = 0.1$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Simulation details: task where temporal credit assignment is difficult (Figure 3)

This task was inspired by the task performed by rodents in [15]. Each trial was split into three periods: the cue period, the delay period, and the decision period. During the cue period, the subject was stimulated with 7 successive binary cues (“left” or “right”), and had to take a corresponding binary decision (“left” or “right”) during the decision period. The trial was considered a success if the decision matched the side that was most often indicated by the 7 cues. No action was required during the delay period. Each cue lasted for 100 ms and the cues were separated by 50 ms. The duration of the delay was distributed uniformly between 500 ms and 1500 ms, and the decision period lasted for 150 ms. The results from Figure 3 were averaged over 20 different runs, where each run was characterized by a different random seed. For each run this resulted in different initialization of the weight matrices and broadcast weights as well as different realizations of random inputs.

Details of the network model and input scheme: We used an LSNN that consisted of 50 LIF neurons and 50 ALIF neurons. All neurons had a membrane time constant of $\tau_m = 20$ ms, a baseline threshold of $v_{\text{th}} = 0.6$, and a refractory period of 5 ms. The time constant of the threshold adaptation was set to $\tau_a = 2000$ ms, and its impact on the threshold was given as $\beta = 1.74 \cdot 10^{-2}$.

Input to this network was provided by 4 populations of 10 neurons each. The first two input populations encoded the cues as follows: When a cue indicated the “left” side (resp. the “right” side), all the neurons within the first (resp. the second) population produced Poisson spike trains with a firing rate of 40 Hz. The third input population spiked randomly throughout the decision period with a firing rate of 40 Hz and was silent otherwise. All the neurons in the last input population produced Poisson spike trains of 10 Hz throughout the trial, which was useful in particular to avoid that the network becomes quiescent during the delay.

Details of the learning procedure: For learning, we used e-prop for classification tasks, see Supplementary Note 3. The target label $\pi_k^{*,t}$ was given as the correct output during the decision

period at the end of a trial. To help the network solve the task, we used a curriculum with an increasing number of cues. We first trained with a single cue, and increased the number of cues to 3, 5 and finally 7. The number of cues increased each time the network achieved less than 8% error on 512 validation trials. The same criterion was used to stop training once 7 cues were reached.

Independent of the learning algorithm that was used (BPTT, e-prop), a weight update was applied once every 64 trials and the gradients were accumulated during those trials additively. All weight updates were implemented using Adam with default parameters [6] and a learning rate of $5 \cdot 10^{-3}$. In the cases of random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with mean 0 and variance 1. In adaptive e-prop we used L2 weight decay (see Supplementary Note 2) for the broadcast weights B_{jk} and the output weights W_{ji}^{out} with a factor of $c_{\text{decay}} = 0.001$. In addition, firing rate regularization, as described in the Supplementary Note 2, was applied with $c_{\text{reg}} = 1$. and a target firing rate of $f^{\text{target}} = 10$ Hz.

Details of the control experiment in Supplementary Figure 8: For this control experiment, we introduced 1 to 3 additional feedforward (recurrent connections disabled) layers of 50 LIF neurons each before the output neurons. Each layer was connected in an all-to-all fashion to the preceding layer. All neurons had a membrane time constant of $\tau_m = 20$ ms. There was no voltage or firing rate regularization applied to the feedforward layers. We also applied the same curriculum of an increasing number of cues as described in the preceding section. In addition each stage of the curriculum was restricted to 1000 weight updates. All other hyperparameters were identical to the preceding section.

Analysis of information in adaptive firing thresholds: We verified (using 64 runs with 128 trials each for testing) that the firing threshold during the delay (500 ms after the last input cue) contained a substantial part of the decision-relevant information. The correct decision could be decoded by a simple readout network (1 hidden layer consisting of 100 ReLU neurons, trained over 200 iterations with 128 trials each) from the current values of the firing thresholds of ALIF neurons with an accuracy of $95.6\% \pm 1.82\%$ (standard deviation). From low pass filtered spikes of LIF neurons at the end of the recall period it could be decoded with an accuracy of $86.6\% \pm 3.7\%$ (standard deviation).

4 Applying supervised learning with e-prop to artificial neural networks (LSTM networks)

Here we show that e-prop can also be applied to artificial neural networks. We chose long short-term memory (LSTM) networks [16] for this demonstration, whose performance defines the standard for RNNs in machine learning. We demonstrate below that LSTM networks can achieve competitive results on TIMIT when trained with e-prop, followed by details on these simulations. In the following sections, we provide details on the LSTM network model used and on eligibility traces for LSTM units.

Phoneme recognition with LSTM networks and e-prop

In Results, we have used e-prop to train LSNNs on the phoneme recognition task TIMIT (see Figure 2). To test whether e-prop is effective also for artificial neural networks, we applied it to LSTM networks on the very same task in its two flavors of frame-wise classification and sequence transcription.

Supplementary Figure 4 shows that e-prop approximates the performance of BPTT in both versions of TIMIT also for LSTM networks very well. For the more difficult version of TIMIT involving sequence transcription, we trained a feedforward sequence of 3 LSTM networks as in [3].

Simulation details: framewise phoneme recognition task with LSTM networks (Supplementary Figure 4)

The data preparation in the two setups (frame-wise phoneme classification and phoneme sequence recognition) were identical to the LSNN case. They are described in Supplementary Note 3. The details on the network models and training procedures are described next for the two task setups separately.

Details of the network model: We used a bi-directional network architecture [2], where the output of an LSTM network was augmented by the output a second LSTM network that received the input sequence in reverse time order. Each of the two networks consisted of 200 LSTM units.

We used a 61-fold softmax output, one for each class of the TIMIT dataset. The LSTM network had ≈ 0.4 million weights, which matched the number of weights in the LSNN for the same task.

Details of the learning procedure: LSTM networks were trained in the same way as LSNNs, see Supplementary Note 3, except for the following differences in hyperparameters: We decayed the learning rate after every 500 weight updates by a factor of 0.3. For L2 weight decay on output and broadcast weights according to 2 we used a factor of $c_{\text{decay}} = 10^{-3}$. As LSTM units are not spiking, we did not use firing rate regularization.

Simulation details: phoneme sequence recognition with CTC and LSTM networks (Supplementary Figure 4)

We compared e-prop and BPTT on the task and the network architecture used in [3]. As for LSNNs, we employed Connectionist Temporal Classification (CTC) to achieve phoneme sequence recognition (see “Phoneme sequence recognition with CTC” in Supplementary Note 3). This enabled us to train networks on unaligned sequence labeling tasks end to end.

Details of the network model: The neurons were structured into 3 recurrent layers. In each layer there were 250 LSTM units. All neurons in all layers of the RNN were connected to the output layer (see “E-prop with many layers of recurrent neurons” in Supplementary Note 3).

Details of the learning procedure: LSTM networks were trained in the same way as LSNNs, see Supplementary Note 3. In the case of BPTT, we also used the peephole feature in the LSTM network model.

LSTM network model

We used a standard model for LSTM units [16], for which the hidden state at time step t is a one dimensional vector containing only the content of the memory cell c_j^t , such that $\mathbf{h}_j^t \stackrel{\text{def}}{=} [c_j^t]$, and z_j^t is the value of its output. The memory cell can be viewed as a register which supports writing, updating, deleting and reading. These operations are controlled independently for each cell j at each time t by input, forget and output gates (denoted by i_j^t , f_j^t and o_j^t respectively). The new cell state candidate that may replace the cell state c_j^{t-1} at each time step t is denoted $\tilde{c}_j^t$. The input, forget, and output sigmoidal gates as well as the cell state candidate of an LSTM unit j

are defined by the following equations:

$$\dot{c}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{c}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{c}} x_i^t\right) \quad (21)$$

$$\dot{\ell}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{\ell}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{\ell}} x_i^t\right) \quad (22)$$

$$\dot{o}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{o}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{o}} x_i^t\right) \quad (23)$$

$$\tilde{c}_j^t = \tanh\left(\sum_i W_{ji}^{\text{rec},c} z_i^{t-1} + \sum_i W_{ji}^{\text{in},c} x_i^t\right), \quad (24)$$

where all the weights used here are parameters of the model (we also used biases that were omitted for readability). Using these notations, one can now write the update of the state of an LSTM unit j in a form that we can relate to our general formalism:

$$c_j^t = \dot{\ell}_j^t c_j^{t-1} + \dot{c}_j^t \tilde{c}_j^t \quad (25)$$

$$z_j^t = \dot{o}_j^t \tanh(c_j^t). \quad (26)$$

In terms of the computational graph in Figure 6, Supplementary Equations (25) and (26) define $M(c_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$ and $f(c_j^t, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$.

Eligibility traces for LSTM units

Eligibility traces for LIF neurons and ALIF neurons were derived in “Derivation of eligibility traces for concrete neuron models” in Methods. Here, we derive eligibility traces for the weights of LSTM units.

To obtain the eligibility traces, we note that the state dynamics of an LSTM unit is given by: $\frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} = \frac{\partial c_j^t}{\partial c_j^{t-1}} = \dot{\ell}_j^t$. For each weight $W_{ji}^{A,B}$ with A being either “in” or “rec” and B being $\dot{c}$, $\dot{\ell}$, or c , we compute a set of eligibility traces. For example, the eligibility vectors for the recurrent weights to the input gate $W_{ji}^{\text{rec},\dot{c}}$, are updated according to Equation (14) in Methods, leading to:

$$\epsilon_{ji}^{\dot{c},t} = \dot{\ell}_j^t \epsilon_{ji}^{\dot{c},t-1} + \tilde{c}_j^t \dot{c}_j^t (1 - \dot{c}_j^t) z_i^{t-1}, \quad (27)$$

resulting in eligibility traces:

$$e_{ji}^{\dot{c},t} = \dot{o}_j^t \tanh'(c_j^t) \epsilon_{ji}^{\dot{c},t}, \quad (28)$$

where $\tanh'(c_j^t) = 1 - \tanh^2(c_j^t)$ is the derivative of $\tanh$. Similarly, the eligibility traces for the input weights to the input gate are obtained by replacing z_i^{t-1} with x_i^t .

Output gates: The gradients with respect to the parameters of the output gate do not require additional eligibility traces. This is because the output gate contributes to the observable state but not to the hidden state, see Supplementary Equations (25) and (26). Therefore, one can use the standard factorization of the error gradient as used in BPTT. For the recurrent weights to the output gates $W_{ji}^{\text{rec},\dot{o}}$, the gradient is given by:

$$\frac{dE}{dW_{ji}^{\text{rec},\dot{o}}} = \sum_t \frac{dE}{dz_j^t} \frac{\partial z_j^t}{\partial W_{ji}^{\text{rec},\dot{o}}} = \sum_t \frac{dE}{dz_j^t} \tanh'(c_j^t) \dot{o}_j^t (1 - \dot{o}_j^t) z_i^{t-1}. \quad (29)$$

Hence, when applying e-prop to the weights of the output gate of an LSTM unit, we use, similarly as in Equation (27) from Methods, the same approximation $\frac{\partial E}{\partial z_j^t}$ of the ideal learning signal $\frac{dE}{dz_j^t}$. The other term is local, depending only on t and $t-1$, and does not require eligibility traces. For input weights to the output gate $W_{ji}^{\text{in},\dot{o}}$, the gradient is obtained by replacing z_i^{t-1} with x_i^t .

5 Reward-based e-prop: Application of e-prop to deep RL

Synaptic plasticity rules for reward-based e-prop

Here, we derive the synaptic plasticity rules that result from gradients of the loss function E , as given in Methods in Equation (32), see Figure 4b for the network architecture. As a result of the general actor-critic framework with policy gradient, this loss function additively combines the loss function for the policy E_π (actor) and the value function E_V (critic).

We consider two cases: First, a simplified case where in each trial, one action is taken at the end of the trial. This is the setup of the reward-based version of the task where temporal credit assignment is difficult of Figure 3, see Supplementary Figure 5 for performance results. Second, we analyse a more general case where actions are taken throughout the trial. This is the setup of the Atari tasks (Figure 4, Figure 5). For both cases, we derive the gradients for the parts E_π and E_V of the loss function E , and express the plasticity rules resulting from these gradients.

Single action at the end of the trial (Supplementary Figure 5): In this setup, a discrete action $a \in \{1, \dots, K\}$ from a set of K possibilities needs to be taken at the last time step T of a trial, leading to a binary-valued reward r^T . As a result, the return R^T (denoted here for notational simplicity as R) is equal to r^T . We assume that the agent chooses action k with probability $\pi_k = \text{softmax}_k(y_1^T, \dots, y_K^T) = \exp(y_k^T) / \sum_{k'} \exp(y_{k'}^T)$. Following the definition of E_π in (30) in Methods one can write the loss function E_π using the one-hot encoded action $\mathbb{1}_{a=k}$ which assumes a value of 1 only if $a = k$ and is 0 otherwise. Hence we can write $\log \pi(a|\mathbf{y}) = \sum_k \mathbb{1}_{a=k} \log \pi_k$, where although we sum over all possible actions, only the term corresponding to the action a that was taken is non zero. Therefore, we can write E_π as:

$$E_\pi = -R \log \pi(a|\mathbf{y}) = -R \sum_k \mathbb{1}_{a=k} \log \pi_k. \quad (30)$$

Interestingly, in the discrete action case, the loss function E_π is reminiscent of the one used for supervised classification, see Supplementary Equation (19) above. But it exhibits two differences: firstly, the indicator of the selected action $\mathbb{1}_{a=k}$ replaces the target label π_k^* , and secondly, the loss is multiplied by the reward R . Hence, the derivation of the weight update is almost identical with the additional difference that we also replace R with $R - V$ in the resulting gradient to reduce the variance of the loss gradient estimate as derived in the general case in (35) from Methods. Finally, in order to optimize E defined in (32) in Methods, we also need to consider $E_V = \frac{1}{2}(R - V)^2$, for which we can reuse the weight update of e-prop for regression given above in (17). Combining the weight updates for E_π and E_V into a single one, we have:

$$\Delta W_{ji}^{\text{rec}} = -\eta \underbrace{\left[(R - V) \sum_k B_{jk}^\pi (\pi_k - \mathbb{1}_{a=k}) - c_V (R - V) B_j^V \right]}_{L_j} \tilde{e}_{ji}^T, \quad (31)$$

where we denote with B_{jk}^π the broadcast weights from output neurons y_k , and with B_j^V the broadcast weights from the output neuron that produces the value prediction V . The choice of these broadcast weights then defines which variant of reward-based e-prop is employed (reward-based symmetric e-prop, reward-based adaptive e-prop, or reward-based random e-prop).

For the synaptic connections of output neurons, the loss gradient can be computed directly from the loss function (32) in Methods. We also subtract the value prediction to reduce variance of the gradient estimate as in (35) in Methods, and obtain for the update rules: $\Delta W_{kj}^{\pi, \text{out}} = -\eta(R - V)(\pi_k - \mathbb{1}_{a=k})\mathcal{F}_\kappa(z_j^T)$, and $\Delta W_j^V = \eta c_V (R - V)\mathcal{F}_\kappa(z_j^T)$. Similarly, the updates of the biases of output neurons are: $\Delta b_k^{\pi, \text{out}} = -\eta(R - V)(\pi_k - \mathbb{1}_{a=k})$, and $\Delta b^V = \eta c_V (R - V)$.

Actions throughout the trial (Figure 4 and 5): In this setup, we assume that the agent can take discrete actions a_k among K possible choices at certain decision times $t_0, \dots, t_n, \dots$

We also assume that the agent chooses action k with a probability $\pi_k^{t_n}$ based on a categorical distribution that depends on the network output: $\pi_k^{t_n} = \pi(a_k | \mathbf{y}^{t_n}) = \text{softmax}_k(y_1^{t_n}, \dots, y_K^{t_n}) = \exp(y_k^{t_n}) / \sum_{k'} \exp(y_{k'}^{t_n})$.

To derive the gradient estimate resulting from reward-based e-prop, we first consider the regression problem defined by the loss function E_V , and note that a major difference to the previous case is that the return R^t integrates future rewards that may arrive long after an action was taken. We begin with the result for regression given above in Supplementary Equation (17). Substituting the relevant variables, we obtain an estimation of the loss gradient:

$$\widehat{\frac{dE_V}{dW_{ji}^{\text{rec}}}} = - \sum_{t'} (R^{t'} - V^{t'}) W_j^{V, \text{out}} \bar{e}_{ji}^{t'}, \quad (32)$$

where $W_j^{V, \text{out}}$ are the weights of the output neuron V_j^t predicting the value function $\mathbb{E}[R^t]$. In order to overcome the obstacle that an evaluation of the return $R^{t'}$ requires to know future rewards, we introduce temporal difference errors $\delta^t = r^t + \gamma V^{t+1} - V^t$, and use that $R^{t'} - V^{t'}$ is equal to the sum $\sum_{t \geq t'} \gamma^{t-t'} \delta^t$. We then reorganize the two sums over t and t' (note that the interchange of the summation order amounts to the equivalence between forward and backward view of RL [17]):

$$\widehat{\frac{dE_V}{dW_{ji}^{\text{rec}}}} = - \sum_{t'} \left(\sum_{t \geq t'} \gamma^{t-t'} \delta^t \right) W_j^{V, \text{out}} \bar{e}_{ji}^{t'} \quad (33)$$

$$= - \sum_t \delta^t \sum_{t' \leq t} \gamma^{t-t'} W_j^{V, \text{out}} \bar{e}_{ji}^{t'} \quad (34)$$

$$= - \sum_t \delta^t \mathcal{F}_\gamma(W_j^{V, \text{out}} \bar{e}_{ji}^t). \quad (35)$$

For the part E_π in the loss function E defined at Equation (30) in Methods, we consider the estimator $\widehat{\frac{\partial E}{\partial z_j^t}}$ given in (35) in Methods, and use our previous definition that the probabilities of actions a_k follow a categorical distribution, using a softmax of the network output y_k . The estimator then becomes:

$$\widehat{\frac{\partial E_\pi}{\partial z_j^t}} = \sum_k W_{kj}^{\pi, \text{out}} \sum_{\{n \mid t_n \geq t\}} \kappa^{t_n - t} (R^{t_n} - V^{t_n}) (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}), \quad (36)$$

where $W_{kj}^{\pi, \text{out}}$ are the weights onto the output neurons y_k^t defining the policy π , and κ is the constant of the low-pass filtering of the output neurons. Following a derivation similar to Supplementary Equations (15) to (17) given above for supervised regression, we arrive at an estimation of the loss gradient of the form:

$$\widehat{\frac{dE_\pi}{dW_{ji}^{\text{rec}}}} = \sum_t \widehat{\frac{\partial E_\pi}{\partial z_j^t}} e_{ji}^t \quad (37)$$

$$= \sum_{t,k} W_{kj}^{\pi, \text{out}} \sum_{\{n \mid t_n \geq t\}} (R^{t_n} - V^{t_n}) (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}) \kappa^{t_n - t} e_{ji}^t \quad (38)$$

$$= \sum_{n,k} (R^{t_n} - V^{t_n}) W_{kj}^{\pi, \text{out}} (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}) \underbrace{\sum_{t \leq t_n} \kappa^{t_n - t} e_{ji}^t}_{\bar{e}_{ji}^{t_n}}. \quad (39)$$

Like in the derivation of the gradient of E_V , this formula hides a sum over future rewards in R^{t_n} that cannot be computed online. It is resolved by introducing the backward view as in Equation (35) above. We arrive at the loss gradient:

$$\widehat{\frac{dE_\pi}{dW_{ji}^{\text{rec}}}} = \sum_t \delta^t \mathcal{F}_\gamma \left(\sum_k W_{kj}^{\pi, \text{out}} (\pi_k^t - \mathbb{1}_{a^t=k}) \bar{e}_{ji}^t \right). \quad (40)$$

Importantly, an action is only taken at times $t_0, \dots, t_n, \dots$, hence for all other times, we set the term $\pi_k^t - \mathbb{1}_{a^t=k}$ to zero.

Finally, the gradient of the loss function E is the sum of the gradients of E_π and E_V in Equations (35) and (40) from Supplementary Materials. Application of stochastic gradient descent with a learning rate of η yields the synaptic plasticity rule given in the Equations (36) and (37) in Methods.

Policy entropy regularization: In order to prevent premature convergence and encourage exploration, we add to the loss function of reward-based e-prop an auxiliary loss that maximizes the entropy H of the policy by $E_H = -\sum_t H(\pi_1^t, \dots, \pi_K^t) \stackrel{\text{def}}{=} -\sum_t H^t$. Hence the Equation (35) defining the estimators of the loss gradients $\widehat{\frac{\partial E}{\partial z_j^t}}$ in Methods becomes:

$$\widehat{\frac{\partial E}{\partial z_j^t}} = -\sum_n (R^{t_n} - V^{t_n}) \frac{\partial \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{\partial z_j^t} + c_V \frac{\partial E_V}{\partial z_j^t} + c_H \frac{\partial E_H}{\partial z_j^t}. \quad (41)$$

To exhibit the term $\frac{\partial E_H}{\partial z_j^t}$, we begin with the definition of entropy and find $E_H = \sum_n \sum_k \pi_k^{t_n} \log \pi_k^{t_n} = -\sum_n H^{t_n}$ for the case of actions that probabilities distributed according to categorical distribution:

$$\frac{\partial E_H}{\partial z_j^t} = -\sum_{\{n | t_n \geq t\}} \sum_k \kappa^{t_n-t} W_{kj}^{\pi, \text{out}} \frac{\partial H^{t_n}}{\partial y_k^{t_n}} \quad (42)$$

$$= \sum_{\{n | t_n \geq t\}} \sum_k \kappa^{t_n-t} W_{kj}^{\pi, \text{out}} \pi_k^{t_n} (\log \pi_k^{t_n} + H^{t_n}). \quad (43)$$

And thus, using a similar derivation as in Supplementary Equations (15) to (17), we find the estimate of the gradient of the loss function E_H as:

$$\widehat{\frac{dE_H}{dW_{ji}}} = \sum_t \frac{\partial E_H}{\partial z_j^t} e_{ji}^t = \sum_{n,k} W_{kj}^{\pi, \text{out}} \pi_k^{t_n} (\log \pi_k^{t_n} + H^{t_n}) \bar{e}_{ji}^{t_n}. \quad (44)$$

Hence this adds a term of the form $L_j^{H,t} \bar{e}_{ji}^t$ to the final learning rule of reward-based e-prop exhibited in Equations (36) and (37) in Methods such that it becomes:

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \delta^t \mathcal{F}_\gamma \left(L_j^t \bar{e}_{ji}^t \right) - \eta L_j^{H,t} \bar{e}_{ji}^t \quad \text{with} \quad (45)$$

$$L_j^{H,t} = c_H \sum_k B_{jk}^\pi \pi_k^t (\log \pi_k^t + H^t). \quad (46)$$

Plasticity rule for the output weights and biases: The gradient of E with respect to the output weights can be computed directly from Equation (32) in Methods without the theory of e-prop. However, it also needs to account for the sum over future rewards that is present in the term $R^t - V^t$. Using a similar derivation as above in Supplementary Equations (33)-(35) the plasticity rule for these weights becomes:

$$\Delta W_{kj}^{\pi, \text{out}} = -\eta \sum_t \left[\delta^t \mathcal{F}_\gamma \left((\pi_k^t - \mathbb{1}_{a^t=k}) \mathcal{F}_\kappa(z_j^t) \right) + \pi_k^t (\log \pi_k^t + H^t) \mathcal{F}_\kappa(z_j^t) \right] \quad (47)$$

$$\Delta W_j^{\text{V, out}} = \eta c_V \sum_t \delta^t \mathcal{F}_\gamma \left(\mathcal{F}_\kappa(z_j^t) \right). \quad (48)$$

Similarly, we find the update rules for the biases of the output neurons. We obtain for these: $\Delta b_k^{\pi, \text{out}} = -\eta \sum_t \left[\delta^t \mathcal{F}_\gamma \left((\pi_k^t - \mathbb{1}_{a^t=k}) \right) + \pi_k^t (\log \pi_k^t + H^t) \right]$, and $\Delta b^{\text{V, out}} = \eta c_V \sum_t \delta^t$.

Simulation details: RL version of the task where temporal credit assignment is difficult (Supplementary Figure 5)

The task considered in this experiment was the same as in Figure 3, but while the task was there formulated as a supervised learning, the network is trained here using a reinforcement learning setup. A reward of 1 was given at the end of the trial, if the agent selected the side which had more cues than the other side, otherwise no reward was given. The network model remained the same as in the supervised setup. The result was computed over 10 different runs and is shown in Supplementary Figure 5: The task can be learnt by reward-based e-prop.

Details of the decision process: In the reinforcement learning setup of the task, one binary action formalizes the decision of the agent (“left” or “right”) at the end of the trial. This decision was sampled according to probabilities π_k that were computed from the network output using a softmax operation as described above.

Details of the learning procedure: For learning, we simulated batches of 64 trials, and applied weight changes at the end of each batch. Independent of the learning method, we used Adam to implement the weight update, using gradients that were accumulated in 64 trials using a learning rate of $5 \cdot 10^{-3}$ and default hyperparameters [6]. For random e-prop, we sampled broadcast weights B_{jk} from a Gaussian distribution with a mean of 0 and a variance of 1. To avoid an excessively high firing rate, regularization, as described in Supplementary Note 2, was applied with $c_{\text{reg}} = 0.1$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Simulation details: Atari task (Figures 4 and 5)

Details of the Atari simulator and of the input scheme: For all our experiments we used the Arcade Learning Environment [18] to simulate various Atari games. This simulator takes the actions from the agent as input and advances the dynamics of the game by repeating the same action for 4 times. The simulator then produces a reward feedback along with the new video frame. The shape of the produced video frame is 160 pixels in width and 210 pixels in height and each pixel has 3 values that describes its color. We first converted the colored video frame into a gray-scale video frame and scaled it to a size of 84 by 84. The gray-scale values (assuming values between 0 and 255) were divided by 255. In our experiments, these observations were each given for 5 ms as input $\mathbf{x}^t, \dots, \mathbf{x}^{t+4}$ to the spiking agent.

Details of evaluation: The results reported in Figure 4 and 5 were computed over 5 different runs, where each run was characterized by a different random seed. For each run this resulted in different random initial weights of the agent and different random numbers for sampling the stochastic action. We chose to compare our spiking agents to the results reported in Table S3 of [19] because of the similarity of the learning approach. Note however that there exist some differences in the evaluation protocol:

1. the results reported in [19] are given as the 5 best runs out of 50, whereas we report the average over 5 runs;
2. their evaluation protocol involves human starting conditions, meaning that evaluation episodes are initialized with human game play, whereas we start episodes randomly with up to 30 No-op actions.

Details of the network model: The network architecture of the agent consisted of a spiking CNN followed by an LSNN. The spiking CNN consisted of 5512 LIF neurons for Pong (10928 LIF neurons for Fishing Derby respectively). These neurons were organized in two layers. Neurons in the first layer received input current resulting from a convolution of the video frame. The

spikes of these neurons were propagated to a second layer of neurons using synapses with a convolutional weight structure. Biases were channel-specific for both convolutions, see Table 1 for all corresponding hyperparameters. All neurons in the two convolutional layers had a membrane time constant of $\tau_{m,\text{CNN}}$ ms and a threshold of $v_{\text{th,CNN}}$, see Table 1. We were forced to use a very high rate of input frames in order to keep the compute time of the learning experiments within the range of our computing resources. Therefore a new image was presented to the network every 5 ms. This forced us to set the membrane constants of the spiking neurons in the preprocessing CNN to the unbiologically low value of 1 ms. We conjecture that a simulation of the same learning processes with larger computing resources, where the presentation time of input frames can be multiplied with some factor F , say $F = 20$, the spiking CNN would be able to process these visual inputs with neurons whose membrane time constants are multiplied with a corresponding factor F , thereby bringing them into a biological range.

The LSNN consisted of 240 LIF neurons and 160 ALIF neurons for Pong (180 LIF neurons and 120 ALIF neurons for Fishing Derby respectively) and received spiking input from all neurons of the second convolutional layer. Membrane time constants τ_m , baseline thresholds v_{th} , adaptation time constants τ_a and adaptation strengths β are reported in Table 1. The refractory period was set to 5 ms. All synaptic delays in the LSNN were 1 ms. The output $\mathbf{y}^t, V^t$ was defined as the membrane voltage of output neurons, whose membrane time constants τ_{out} are reported in Table 1. These output neurons received spikes from all neurons in the network, including neurons in the convolutional layers.

Details of the action set and of the action generation: The set of valid actions was selected according to the game at hand. For Pong, we used a set of 3 valid actions: Up, Down and Stay (No op). For Fishing Derby, we used 9 possible action choices: Stay (No op), Fire, Up, Right, Left, Down, Up-right, Up-Left, Down-right and Down-left. Actions were sampled according to a discrete probability distribution, where the k -th action was sampled with a probability of $\pi_k^t = \text{softmax}_k(y_1^t, \dots, y_K^t)$. Since the agent received the same video frame for 5 ms, actions were also generated only after every 5 ms.

Details of the learning procedure: The agent was trained consuming a total of $250 \cdot 10^6$ action generations, corresponding to 10^9 video frames of the Atari simulator (actions are repeated 4 times internally). We computed gradients resulting from reward-based symmetric e-prop according to Supplementary Equation (45). A common learning scheme is to use actors living in parallel copies of the environment that can regularly communicate gradients with each other during ongoing episodes [19]. But this scheme can hardly provide a model for learning in the brain. For this reason, we designed a schedule of episodes of increasing length and performed weight updates only after 32 completed episodes (64 for Fishing Derby, respectively). A single agent can implement this algorithm through local synaptic plasticity. However this scheme is equivalent to simulating these 32 independent episodes in parallel -but without communication between learning agents- instead of sequentially. This observation can be used to save simulation time.

For Pong the weights of the CNN are only trained to minimize the regularization of the neural activity, hence it does not require learning signals. For Fishing Derby, as every neuron is connected to the output neurons, we define broadcast weights $B_{jk}^{\pi,\text{CNN}}, B_{jk}^{V,\text{CNN}}$ for neurons in the CNN to be symmetric to the output weights: $B_{jk}^{\pi,\text{CNN}} = W_{kj}^{\pi,\text{out,CNN}}, B_j^{V,\text{CNN}} = W_j^{V,\text{out,CNN}}$.

Due to simulation constraints, we reset the fading memory filter $F_\gamma(L_j^t \bar{e}_{ji}^t)$ as well as the eligibility vectors ϵ_{ji}^t to zero after every 500 ms for Pong (150 ms for Fishing Derby respectively). The weight updates were applied using Adam [6] with a learning rate of 10^{-3} (The parameter ϵ was set to 1). In Fishing Derby we clipped these gradients if exceeding a norm of 1000. The maximum episode length T_{max} was restricted to 1000 ms at the start of training, and was increased after every 8750 s by ΔT , see Table 1. We used a discount factor of $\gamma = 0.998$. Note that an action is generated only after every 5 ms. The coefficient c_V that trades off between E_π and E_V can be found in Table 1. We also applied entropy regularization as in Supplementary Equation (44) to prevent premature convergence to suboptimal policies. The value of the coefficient c_H that

adds this loss E_H to the objective is provided in Table 1. To avoid an excessively high firing rate, regularization was applied as described in 2. The target firing rate was given by $f^{\text{target}} = 20$ Hz. We used further regularization to prevent membrane voltages from assuming values for which ψ_j is zero.

6 Evaluation of four variations of *e-prop* (Supplementary Figure 7)

We evaluate here the performance of four variations of random e-prop. In these variations, we used

- truncated eligibility traces for LIF neurons,
- global broadcast weights,
- temporally local broadcast weights, and
- a replacement of the eligibility trace by the corresponding term of the Clopath rule,

respectively. The considered task, whose implementation details are described below 6, is an extension of the task used in [1]. In this task, an RSNN was trained to autonomously generate a 3 dimensional target signal for 1 second. Each dimension of the target signal was given by the sum of four sinusoids with random phases and amplitudes. Similar to [1], the network received a clock input that indicated the current phase of the pattern.

In Supplementary Figure 7a, we show the spiking activity of a randomly chosen subset of 20 out of the 600 neurons in the RSNN along with the output of the three output neurons after application of random e-prop for 1, 100, and 500 seconds, respectively. In this representative example, the network achieved a very good fit to the target signal (normalized mean squared error 0.01).

A truncated eligibility trace for LIF neurons

A replacement of the term $\bar{z}_i^t$ with z_i^t in Equation (23) from Methods yields a performance that is reported in panel b of Supplementary Figure 7 as “Trunc. e-trace”. Its performance is for the considered task only slightly worse than that of random e-prop.

Global broadcast weights

Since 3-factor rules have primarily been studied so far with a global 3rd factor, we asked how the performance of e-prop would change if the same broadcast weight would be used for broadcast connections between all output neurons k and network neurons j . We set this global broadcast weight equal to $\frac{1}{\sqrt{n}}$ with n being the number of neurons in the network. Figure 2 and Supplementary Figure 2 show together that the performance for the considered task is much worse than that of random e-prop. We have also tested this on TIMIT with LSNNs and found there an increase of the frame-wise error rate from 36.9% to 52% when replacing the broadcast weights of random e-prop with a global one. On the harder version of same task, the error rate at the sequence level increased from 34.7% to 60%.

Temporally local broadcast weights

One can train RNNs also by applying the broadcast alignment method of [20] and [21] for feed-forward networks to the unrolled version (see Figure 1b) of the RNN. In contrast to e-prop, this approach suggests to draw new random broadcast weights for each layer of the unrolled network, i.e., for each time step of the RNN. Figure 7c shows that this variation of random e-prop performs much worse. However an intermediate version where the random broadcast weights are redrawn every 20 ms performs about equally well as random e-prop for the considered task.

Replacing the eligibility trace by the corresponding term of the Clopath rule

The dependence of the synaptic plasticity rules from e-prop on the postsynaptic membrane potential through the pseudo-derivative in the eligibility traces yields some similarity to some previously proposed rules for synaptic plasticity, such as that of [5], which were motivated by experimental data on the dependence of synaptic plasticity on the postsynaptic membrane potential. We therefore tested the performance of random e-prop, where the eligibility trace was replaced by the corresponding term from the “Clopath rule”:

$$[v_j^t - v_{\text{th}}^+]^+ [\bar{v}_j^t - v_{\text{th}}^-]^+ \bar{z}_i^{t-1}, \quad (49)$$

where $\bar{v}_j^t$ is an exponential trace of the post synaptic membrane potential, with a time constant of 10 ms chosen to match their data. $[\cdot]^+$ is the rectified linear function. The thresholds v_{th}^- and v_{th}^+ were $\frac{v_{\text{th}}}{4}$ and 0 respectively. Supplementary Figure 7b shows that the resulting synaptic plasticity rule performed quite well.

Simulation details: pattern generation task

The performance in this task is reported as a normalized mean squared error (nmse) that we defined for this task as: $\text{nmse} = \frac{\sum_{t,k} (y_k^t - y_k^{*,t})^2}{\sum_{t,k} (y_k^{*,t} - \bar{y}_k^*)^2}$, where we set $\bar{y}_k^* = \frac{1}{T} \sum_t y_k^{*,t}$.

Details of the network model and of the input scheme: We used a network that consisted of 600 LIF neurons. Each neuron had a membrane time constant of $\tau_m = 20$ ms and a refractory period of 3 ms. The firing threshold was set to $v_{\text{th}} = 0.41$. Output neurons used a membrane time constant of $\tau_{\text{out}} = 20$ ms. The network received input from 20 input neurons, divided into 5 groups, which indicated the current phase of the target sequence similar to [1]. Neurons in group $i \in \{0, 4\}$ produced 100 Hz regular spike trains during the time interval $[200 \cdot i, 200 \cdot i + 200)$ ms and were silent at other times.

Details of the target pattern: The target signal had a duration of 1000 ms and each component was given by the sum of four sinusoids, with fixed frequencies of 1 Hz, 2 Hz, 3 Hz, and 5 Hz. At the start of learning, the amplitude and phase of each sinusoid in each component was drawn uniformly in the range $[0.5, 2]$ and $[0, 2\pi]$ respectively. This signal was not changed afterwards.

Details of the learning procedure: For learning, we computed gradients after every 1 second of simulation, and carried out the weight update using Adam [6] with a learning rate of $3 \cdot 10^{-3}$ and default hyperparameters. After every 100 iterations, the learning rate was decayed by a factor of 0.7. For random e-prop, the broadcast weights B_{jk} were sampled from a Gaussian distribution with a mean of 0 and a variance of $\frac{1}{n}$, where n is the number of network neurons.

Firing rate regularization, as described in Supplementary Note 2, was applied with $c_{\text{reg}} = 0.5$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Supplementary References

- [1] Nicola, W. & Clopath, C. Supervised learning in spiking neural networks with force training. *Nat. Comm.* (2017).
- [2] Graves, A. & Schmidhuber, J. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks* (2005).
- [3] Graves, A., Mohamed, A.-R. & Hinton, G. Speech recognition with deep recurrent neural networks. *ICASSP* (2013).

- [4] Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R. & Schmidhuber, J. LSTM: A search space odyssey. *IEEE TNNLS* (2017).
- [5] Clopath, C., Büsing, L., Vasilaki, E. & Gerstner, W. Connectivity reflects coding: a model of voltage-based STDP with homeostasis. *Nat. Neuro.* (2010).
- [6] Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *Preprint at <https://arxiv.org/abs/1412.6980>* (2014).
- [7] Bellec, G. *et al.* Biologically inspired alternatives to backpropagation through time for learning in recurrent neural nets. *Preprint at <http://arxiv.org/abs/1901.09049>* (2019).
- [8] Kappel, D., Legenstein, R., Habenschuss, S., Hsieh, M. & Maass, W. A dynamic connectome supports the emergence of stable computational function of neural circuits through reward-based learning. *eNeuro* (2018).
- [9] Bellec, G., Kappel, D., Maass, W. & Legenstein, R. Deep rewiring: Training very sparse deep networks. *ICLR* (2018).
- [10] Glass, J., Smith, A. & K. Halberstadt, A. Heterogeneous acoustic measurements and multiple classifiers for speech recognition (1999).
- [11] Amodei, D. *et al.* Deep speech 2 : End-to-end speech recognition in english and mandarin. *ICML* **48**, 173–182 (2016).
- [12] Graves, A., Fernández, S., Gomez, F. & Schmidhuber, J. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *ICML* 369–376 (2006).
- [13] Graves, A. Supervised sequence labelling. In *PhD Thesis from TU Munich* (2012).
- [14] Marszałek, M., Laptev, I. & Schmid, C. Actions in context. *CVPR* (2009).
- [15] Morcos, A. S. & Harvey, C. D. History-dependent variability in population dynamics during evidence accumulation in cortex. *Nat. Neuro.* (2016).
- [16] Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Computation* **9**, 1735–1780 (1997).
- [17] Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (MIT press, 2018).
- [18] Bellemare, M. G., Naddaf, Y., Veness, J. & Bowling, M. The arcade learning environment: An evaluation platform for general agents. *JAIR*, DOI: [<http://doi.org/10.1613/jair.3912>] **47**, 253–279 (2013).
- [19] Mnih, V. *et al.* Asynchronous methods for deep reinforcement learning. In *ICML*, 1928–1937 (2016).
- [20] Lillicrap, T. P., Cownden, D., Tweed, D. B. & Akerman, C. J. Random synaptic feedback weights support error backpropagation for deep learning. *Nat. Comm.* (2016).
- [21] Nøkland, A. Direct feedback alignment provides learning in deep neural networks. In *NIPS* (2016).